

Modelos Avanzados de Computacion



*Escuela Técnica Superior de Ingenierías
Informática y de Telecomunicación*

Los Del DGIIM, losdeldgiim.github.io

Doble Grado en Ingeniería Informática y Matemáticas
Universidad de Granada



Esta obra está bajo una Licencia Creative Commons Atribución-NoComercial-SinDerivadas 4.0 Internacional (CC BY-NC-ND 4.0).

Eres libre de compartir y redistribuir el contenido de esta obra en cualquier medio o formato, siempre y cuando des el crédito adecuado a los autores originales y no persigas fines comerciales.

Modelos Avanzados de Computacion

Los Del DGIIM, losdeldgiim.github.io

José Manuel Sánchez Varbas

Granada, 2026

Índice general

1. Ejercicios de Decibilidad	5
1.1. Soluciones	8
2. Ejercicios sobre Clases de Complejidad	33
2.1. Soluciones	35

1. Ejercicios de Decibilidad

Ejercicio 8. Demostrar que el problema de la parada, determinar el conjunto de parejas (M, w) tales que la MT M para cuando tiene a w como entrada, es semidecidible pero no decidible.

Ejercicio 9. (MANUAL) Demostrar que determinar si una MT acepta al menos un palíndromo no es decidible pero sí semidecidible.

Ejercicio 10. Supongamos que tenemos MT con dos tipos de estados: *campana* y *silbato*. Determinar que saber si una MT entrará alguna vez en un estado *silbato* es indecidible. ¿Es semidecidible?

Ejercicio 11. (MANUAL) Demostrar que es indecidible (no recursivo) saber si una MT termina escribiendo un 1 cuando comienza con una cinta completamente en blanco.

Ejercicio 12. Determinar si los siguientes problemas son decidibles, semidecidibles o no semidecidibles:

- a) Determinar si el lenguaje de una MT contiene, al menos, dos palabras distintas.
- b) Determinar si el lenguaje reconocido por una MT es finito.
- c) (MANUAL) Determinar si el lenguaje reconocido por una MT es infinito.
- d) Determinar si el lenguaje de una MT es independiente del contexto.

Ejercicio 13. Sea L el lenguaje formado por pares de códigos de MT más un entero (M_1, M_2, k) tales que $L(M_1) \cap L(M_2)$ contiene, al menos, k palabras. Demostrar que L es semidecidible pero no decidible.

Ejercicio 14. Demostrar que los siguientes lenguajes son recursivos:

- a) El conjunto de las MT M tales que al comenzar con la cinta en blanco, en algún momento escribirán un símbolo no blanco en la cinta.
- b) El conjunto de las MT que nunca se mueven a la izquierda.
- c) El conjunto de los pares (M, w) tales que M , al actuar sobre la entrada w , nunca lee una casilla de la cinta más de una vez.

Ejercicio 15. Indicar si los siguientes problemas son decidibles, semidecidibles o no semidecidibles:

- a) (MANUAL) Determinar si una MT se para para cualquier entrada.
- b) Determinar si una MT no se para para ninguna entrada.
- c) Determinar si una MT se para, al menos, para una entrada.
- d) Determinar si una MT no se para, al menos, para una entrada.

Ejercicio 16. Supongamos que tenemos 4 lenguajes L_1 , L_2 , L_3 , L_4 de tal manera que existe una reducción de L_1 a L_2 , de L_2 a L_3 y de L_4 a L_3 . Para cada una de las siguientes afirmaciones, indicar si para todos los casos son CIERTAS, para todos los casos son FALSAS o si son POSIBLES (en algunos casos son ciertas y, en otros, son falsas).

- a) L_1 es recursivamente enumerable pero no recursivo, y L_3 es recursivo.
- b) L_1 no es recursivo y L_4 no es recursivamente enumerable.
- c) El complementario de L_1 no es recursivamente enumerable, pero el complementario de L_2 es recursivamente enumerable.
- d) El complementario de L_2 no es recursivo, pero el complementario de L_3 es recursivo.
- e) Si L_1 es recursivo, entonces el complementario de L_2 es recursivo.
- f) Si L_3 es recursivo, entonces el complementario de L_4 es recursivo.
- g) Si L_3 es recursivamente enumerable, entonces la unión de L_2 y L_4 es recursivamente enumerable.
- h) Si L_3 es recursivamente enumerable, entonces la intersección de L_2 y L_4 es recursivamente enumerable.

Ejercicio 17. Sea el problema de determinar si una máquina de Turing acepta a lo más 100 palabras. Determinar si es decidible, semidecidible o no semidecidible. Justificar la respuesta.

Ejercicio 18. Determinar cuáles de los siguientes problemas son decidibles, semidecidibles o no semidecidibles:

- a) Saber si una MT para una entrada 0011 no va a usar más de 10 casillas de la cinta.
- b) Dadas dos MT saber si aceptan el mismo lenguaje

Ejercicio 19. Determinar cuáles de los siguientes problemas son decidibles, semidecidibles o no semidecidibles:

- a) Dada una MT M y una palabra u , saber si la MT acepta la palabra u en un número de pasos menor o igual a $|u|$.
- b) Dada una MT determinar si no acepta la palabra vacía.

Ejercicio 20. Determinar cuáles de los siguientes problemas son decidibles, semi-decidibles o no semidecidibles (se supone que las MTs tienen a $\{0, 1\}$ como alfabeto de entrada):

- a) Dadas dos máquinas de Turing, M_1 y M_2 , determinar si existe, al menos, una palabra aceptada simultáneamente por ambas máquinas.
- b) (MANUAL) Dada una MT, determinar si para toda palabra de entrada no realiza más de 5 movimientos.
- c) Determinar si una MT no es determinista.
- d) (MANUAL) Dada una MT, determinar si ninguna de las palabras que acepta es un palíndromo.

Justifica las respuestas.

Ejercicio 21. Se considera el siguiente problema: dadas dos máquinas de Turing, determinar si aceptan el mismo lenguaje. Razonar si este problema es decidible, semidecidible o no semidecidible.

Ejercicio 22. Determinar, justificando las respuestas, cuáles de los siguientes problemas son decidibles, semidecidibles o no semidecidibles:

- a) Dada una máquina de Turing y una palabra de entrada determinar si visita menos de 10 casillas distintas de la cinta de lectura.
- b) Dadas dos Máquinas de Turing, determinar si el conjunto de las palabras aceptadas a la vez por ambas máquinas de Turing es finito.

Ejercicio 23. Indica cuáles de los siguientes problemas son decidibles, semidecidibles o no semidecidibles:

- a) Determinar si el lenguaje aceptado por una MT es regular.
- b) Dadas dos MTs determinar si hay una palabra que es aceptada por las dos MT y además es un palíndromo.

Ejercicio 24. Indica cuáles de los siguientes problemas son decidibles, semidecidibles o no semidecidibles:

- a) Dada una MT determinar si todos sus estados son accesibles (se puede llegar a ellos para un cálculo para alguna entrada).
- b) Dada una MT determinar si su número de estados es menor o igual a 5.

Ejercicio 25. Demuestra que el Problema de la Correspondencia de Post con un alfabeto A que tiene un sólo elemento es decidible.

1.1. Soluciones

Ejercicio 1.1.8. Demostrar que el problema de la parada, determinar el conjunto de parejas (M, w) tales que la MT M para cuando tiene a w como entrada, es semidecidible pero no decidible.

Definimos $\text{PARADA}(M, w) = \{\langle M, w \rangle : M \text{ para con entrada } w\}$. Para ver que es semidecidible, la idea es construir un simulador de MT M_p , que lea la codificación $\langle M, w \rangle$, simule la MT M sobre la entrada w y si en algún momento de la simulación M para, M_p acepta, de manera que M_p acepte exactamente las entradas $\langle M, w \rangle$ tal que M para con w .

Algoritmo 1 Máquina de Turing M_p

Entrada: Una palabra $v \in \{0, 1\}^*$
 Comprobar si v es de la forma $\langle M, w \rangle$
Si v no es una codificación válida **entonces**
 Rechazar
Fin Si
 Simular paso a paso la ejecución de M sobre la entrada w
Si en algún momento la simulación de M para **entonces**
 Aceptar
Fin Si

Esta MT M_p acepta exactamente los casos positivos de las palabras del lenguaje L_p asociado:

- Si $\langle M, w \rangle \in \text{PARADA}$, la simulación de M sobre w termina en un número finito de pasos, y M_p aceptará.
- Si $\langle M, w \rangle \notin \text{PARADA}$, entonces M cicla indefinidamente sobre w , y la simulación hace continúa también indefinidamente, por tanto M_p no aceptará.

Por lo tanto, PARADA es semidecidible.

Para ver que no es decidible, la idea será hacer una reducción de universal a parada, que sabemos que es semidecidible, pero no decidible por teoría. Consideramos los problemas:

$$\begin{cases} P1 = & \text{UNIVERSAL : dada una MT } M \text{ y una palabra } w, \text{ determinar si } M \text{ acepta } w \\ P2 = & \text{PARADA : dada una MT } N \text{ y una palabra } u, \text{ determinar si } N \text{ para con } u \end{cases}$$

Construimos un algoritmo F que transforma cada instancia (M, w) de UNIVERSAL en una instancia (N, u) de PARADA, donde $N = F(M)$ es la MT siguiente:

Esta construcción es algorítmica, ya que a partir de M puede construirse una MT N que simule a M sobre su misma entrada u y que solo pare cuando la simulación llegue a aceptación.

Veamos la equivalencia:

Algoritmo 2 Máquina de Turing $N = F(M)$ **Entrada:** Una palabra $u \in \{0, 1\}^*$

- 1: Simular la ejecución de M sobre la entrada u
- 2: Si M acepta u , entonces N para
- 3: En otro caso, N no para (cicla).

- Si (M, w) es un caso positivo de UNIVERSAL, es decir, si M acepta w , entonces al ejecutar N sobre w la simulación llega a aceptación y N para. Por tanto, $(N, w) \in \text{PARADA}$.
- Si (M, w) es un caso negativo de UNIVERSAL, es decir, si M no acepta w , entonces la máquina N , sobre w , no para. Por tanto, $(N, w) \notin \text{PARADA}$. Así

$$(M, w) \in \text{UNIVERSAL} \iff (N, w) \in \text{PARADA}$$

Por tanto, tenemos que $\text{UNIVERSAL}(M, w) \propto \text{PARADA}(N, w)$. Si PARADA fuera decidable, tendríamos que UNIVERSAL también lo sería porque UNIVERSAL se reduce a PARADA. Sin embargo, sabemos que esto no es cierto, por lo que PARADA no es decidable.

Ejercicio 1.1.9. (MANUAL) Demostrar que determinar si una MT acepta al menos un palíndromo no es decidable pero sí semidecidible.

Dada una MT M , definimos $\text{PAL}(M) = \{\langle M \rangle : \exists w \in \{0, 1\}^* \text{ palíndromo tal que } M \text{ acepta } w\}$.

Para la no decidibilidad, vemos que la propiedad “aceptar al menos un palíndromo” depende solo del lenguaje aceptado por la máquina, $L(M)$, y no de la máquina concreta, es decir, es una propiedad de los lenguajes r.e.. Además, es una propiedad no trivial, ya que el lenguaje $\{11\}$ la cumple, pero $\{10\}$ no. Así, por el Teorema de Rice, PAL no es decidable.

Para la semidecidibilidad, consideramos una MTND M_{PAL} que funciona como sigue:

Algoritmo 3 MTND M_{PAL} **Entrada:** Una codificación $\langle M \rangle$ de una MT M Selecciona de forma no determinista una palabra $w \in \{0, 1\}^*$.Comprueba si w es un palíndromo.Si w es palíndromo, simula M con entrada w Si M acepta w , entonces M_{PAL} acepta.

Tenemos entonces que, si M acepta al menos un palíndromo w_0 , existe una rama de cómputo de M_{PAL} que precisamente toma w_0 , comprueba que es palíndromo y acepta. Sin embargo, si M no acepta ningún palíndromo, entonces ninguna rama puede aceptar. Por lo tanto, M_{PAL} acepta exactamente las codificaciones $\langle M \rangle$ tales que M acepta al menos un palíndromo, de donde $\text{PAL}(M)$ es semidecidible.

Ejercicio 1.1.10. Supongamos que tenemos MT con dos tipos de estados: *campana* y *silbato*. Determinar que saber si una MT entrará alguna vez en un estado *silbato* es indecidible. ¿Es semidecidible?

Definimos

$$\text{SILBATO}(M) = \{\langle M \rangle : \exists w \in \{0, 1\}^* : M \text{ entra alguna vez en un estado silbato con entrada } w\}$$

Para ver que no es decidible, dada una MT M , como la propiedad “entrar en un estado silbato” no es una propiedad del lenguaje $L(M)$, tampoco es una propiedad de los lenguajes r.e., y no puede aplicarse el Teorema de Rice.

Haremos entonces una reducción desde un problema no decidible de tal manera que sea sencilla. Por ejemplo, PARADA. Consideramos entonces el problema

$$\text{PARADA}(M, w) = \{\langle M, w \rangle : M \text{ para con entrada } w\}$$

Vamos a construir un proceso algorítmico F para pasar de una instancia de PARADA a una instancia de SIL. Sea (M, w) una instancia de PARADA, construimos la siguiente máquina de Turing $F(M, w)$ (que es una instancia de SIL)

Algoritmo 4 MT $F(M, w)$

Entrada: $v \in \{0, 1\}^*$

Borramos v de la cinta.

Copiamos $\langle M \rangle 111w$ en la cinta.

Simulamos M con entrada w .

Si M para **entonces**

$F(M, w)$ entra en un estado *silbato*.

Si M no para **entonces**

$F(M, w)$ cicla y nunca entra en un estado *silbato*.

Vemos que la máquina $F(M, w)$ se construye de tal manera que todos los estados usados para las tareas previas a que M termine sean de tipo *campana*. Si M termina, entonces añadimos un nuevo estado especial q_s , de tipo *silbato*, al que se llega únicamente cuando la simulación termina. Por lo tanto

$$\langle M, w \rangle \in \text{PARADA} \iff \langle F(M, w) \rangle \in \text{SILBATO}$$

Puesto que:

1. Si M para con w (caso positivo de PARADA), entonces $F(M, w)$ termina la simulación y entra en q_s (caso positivo de SILBATO).
2. Si M no para con w (caso negativo de PARADA), entonces $F(M, w)$ ciclará y nunca entrará en estado silbato (caso negativo de SILBATO).

Por lo tanto, $\text{PARADA} \leq \text{SILBATO}$. Si SILBATO fuera decidible, también lo sería PARADA, cosa que sería una contradicción. Así, SILBATO no es decidible.

La semidecidibilidad puede comprobarse con una MTND que seleccione de forma no determinista una entrada u y simule M con dicha entrada y acepte en caso de que durante la simulación M entre en un estado *silbato*. Más en detalle:

Algoritmo 5 MTND $M_{SILBATO}$

Entrada: Una codificación $\langle M \rangle$ de una MT M
 Selecciona de forma no determinista una palabra $u \in \{0, 1\}^*$.
 Simulamos M con entrada u .
 Si en algún momento de la simulación M entra en un estado *silbato*, entonces $M_{SILBATO}$ acepta.

Ejercicio 1.1.11. (MANUAL) Demostrar que es indecidible (no recursivo) saber si una MT termina escribiendo un 1 cuando comienza con una cinta completamente en blanco.

Hecho en el manual. Para la no decidibilidad hay que hacer una reducción desde UNIVERSAL al problema, y para la semidecidibilidad basta ejecutar un bucle $i = 1, \dots, \infty$ tal que se simule la MT con entrada ε en i pasos, y si en alguno de esos i pasos escribe un 1 acepta la entrada.

Ejercicio 1.1.12. Determinar si los siguientes problemas son decidibles, semidecidibles o no semidecidibles:

- a) Determinar si el lenguaje de una MT contiene, al menos, dos palabras distintas.

Definimos $DOSP_{al}(M) = \{\langle M \rangle : \exists u, v \in L(M) \text{ con } u \neq v\}$. Vemos que no es decidible, ya que la propiedad “ $L(M)$ contiene al menos dos palabras distintas” depende solo del lenguaje $L(M)$, no de la MT M concreta, luego es una propiedad de los lenguajes r.e.. Además, es no trivial, ya que $\{0\}$ no cumple la propiedad, pero $\{0, 1\}$ sí la cumple. Por el Teorema de Rice, $DOSP_{al}(M)$ es no decidible. Veamos que es semidecidible. Para ello, consideramos una MTND $M_{DOSP_{al}}$ que funciona como sigue:

Algoritmo 6 MTND $M_{DOSP_{al}}$

Entrada: Una codificación $\langle M \rangle$ de una MT M
 Selecciona de forma no determinista dos palabras $u, v \in \{0, 1\}^*$.
 Si $u = v$, entonces $M_{DOSP_{al}}$ rechaza.
 Simulamos M con entrada u y con entrada v .
 Si ambas simulaciones aceptan, entonces $M_{DOSP_{al}}$ acepta.

Tenemos entonces que, si M acepta al menos dos palabras distintas u_0, v_0 , existe una rama de cómputo de $M_{DOSP_{al}}$ que precisamente toma u_0 y v_0 , simula $M(u_0)$ y $M(v_0)$, y como ambas aceptan, esa rama acepta. Sin embargo, si M no acepta dos palabras distintas, entonces para cualquier par distinto (u, v) , al menos una de las dos simulaciones no aceptará y consecuentemente esa rama no aceptará. Por lo tanto, $M_{DOSP_{al}}$ acepta exactamente las codificaciones $\langle M \rangle$ tales que M acepta al menos dos palabras distintas, de donde $DOSP_{al}(M)$ es semidecidible.

- b) Determinar si el lenguaje reconocido por una MT es finito.

Definimos $\text{FINITO}(M) = \{\langle M \rangle : L(M) \text{ es finito}\}$. Vemos que no es decidible, ya que la propiedad “ $L(M)$ es finito” depende solo del lenguaje $L(M)$, no de la MT M concreta, luego es una propiedad de los lenguajes r.e.. Además, es no trivial, ya que cumple la propiedad (una MT sin estado final), pero $\{0, 1\}^*$ no la cumple (una MT con el estado inicial también final que acepte todas las palabras). Por el Teorema de Rice, $\text{FINITO}(M)$ es no decidible.

Para lo no semidecidibilidad se puede hacer una reducción desde

$$\text{DIAGONAL}(M) = \{\langle M \rangle : M \text{ no acepta } \langle M \rangle\}$$

(que sabemos por teoría que no es semidecidible), igual que en el apartado c), pero permutando los casos en que se rechaza por lo que se acepta. El proceso algorítmico F pasará de una instancia de DIAGONAL a una instancia de FINITO . Sea entonces M una instancia de DIAGONAL , y construimos la siguiente máquina de Turing $F(M)$:

Algoritmo 7 MT $F(M)$

Entrada: Una palabra $v \in \{0, 1\}^*$.

Si v no es de la forma 0^n con $n > 0$ **entonces**

$F(M)$ rechaza.

Si $v = 0^n$ para cierto $n > 0$ **entonces**

Simulamos M con entrada $\langle M \rangle$ durante n pasos.

Si M acepta $\langle M \rangle$ **entonces**

$F(M)$ acepta.

Si no entonces

$F(M)$ rechaza.

c) (MANUAL) Determinar si el lenguaje reconocido por una MT es infinito.

Hecho en el manual.

d) Determinar si el lenguaje de una MT es independiente del contexto.

Definimos $\text{INDCONT}(M) = \{\langle M \rangle : L(M) \text{ es independiente del contexto}\}$. Al menos es no decidible por el Teorema de Rice, ya que, para una MT M , la propiedad “ser $L(M)$ independiente del contexto” es una propiedad que depende solo de $L(M)$, no de M , luego es una propiedad no trivial de los lenguajes r.e. (no trivial porque sabemos que existe una MT M_1 tal que $L(M_1)$ es independiente del contexto y M_2 tal que $L(M_2)$ no es independiente del contexto (por Modelos de Computación)). De hecho, podemos ver que es no semidecidible por medio de una reducción desde

$$\text{DIAGONAL}(M) = \{\langle M \rangle : M \text{ no acepta } \langle M \rangle\}$$

Sea M'' una MT fija tal que

$$L(M'') = \{0^n 1^n 0^n : n \geq 0\},$$

que no es un lenguaje independiente del contexto (puede comprobarse con el Lema de bombeo). Dada una MT M , construimos una MT M' con el siguiente comportamiento:

Algoritmo 8 MT M'

Entrada: Una palabra $x \in \{0, 1\}^*$

Simulamos M con entrada $\langle M \rangle$.

Si M acepta $\langle M \rangle$ **entonces**

Simulamos M'' con entrada x .

Si M'' acepta x **entonces**

M' acepta.

Si no

M' rechaza.

Fin Si

Fin Si

Tenemos entonces que:

- Si $\langle M \rangle \in \text{DIAGONAL}$, entonces M no acepta $\langle M \rangle$. Por tanto, al simular M con entrada $\langle M \rangle$, pueden pasar dos cosas: o bien M rechaza, o bien M cicla. En ambos casos, M' no acepta ninguna palabra. Luego

$$L(M') = \emptyset.$$

Como $\emptyset$ es independiente del contexto, se tiene que

$$\langle M' \rangle \in \text{INDCONT.}$$

- Si $\langle M \rangle \notin \text{DIAGONAL}$, entonces M acepta $\langle M \rangle$. Por construcción, en ese caso M' pasa a comportarse como M'' sobre su entrada x . Por tanto,

$$L(M') = L(M'') = \{0^n 1^n 0^n : n \geq 0\}.$$

Como este lenguaje no es independiente del contexto, se tiene que

$$\langle M' \rangle \notin \text{INDCONT.}$$

Así,

$$\langle M \rangle \in \text{DIAGONAL} \iff \langle M' \rangle \in \text{INDCONT.}$$

Por tanto,

$$\text{DIAGONAL} \propto \text{INDCONT.}$$

Como DIAGONAL es no semidecidible, INDCONT es no semidecidible.

Ejercicio 1.1.13. Sea L el lenguaje formado por pares de códigos de MT más un entero (M_1, M_2, k) tales que $L(M_1) \cap L(M_2)$ contiene, al menos, k palabras. Demostrar que L es semidecidible pero no decidable.

Vemos que

$$L = \{(M_1, M_2, k) : |L(M_1) \cap L(M_2)| \geq k\}$$

Algoritmo 9 MTND M_L

Entrada: Una codificación $\langle M_1, M_2, k \rangle$, donde M_1 y M_2 son MT y $k \in \mathbb{Z}$.
Si $k = 0$ **entonces**
 M_L acepta.
Fin Si
 Selecciona de forma no determinista k palabras $u_1, \dots, u_k \in \{0, 1\}^*$.
Si $\exists i, j \in \{1, \dots, k\} : i \neq j \quad \wedge \quad u_i = u_j$ **entonces**
 M_L rechaza.
Fin Si
Para $i = 1, \dots, k$ **hacer**
 Simulamos M_1 con entrada u_i .
 Simulamos M_2 con entrada u_i .
 Si alguna de las dos simulaciones rechaza **entonces**
 M_L rechaza.
 Fin Si
Fin Para
Si todas las simulaciones aceptan **entonces**
 M_L acepta.
Fin Si

Para ver que es semidecidible, construimos la siguiente *MTND*:

Vemos que funciona, ya que $(M_1, M_2, k) \in L$ si y solo si existen k palabras distintas $u_1, \dots, u_k$ tales que $u_i \in L(M_1) \cap L(M_2) \quad \forall i = 1, \dots, k$. Por lo tanto, M_L semidecide L , luego L es semidecidible.

Para ver que L es no decidible, hacemos una reducción desde

$$\text{C-VACIO}(M) = \{\langle M \rangle : L(M) \neq \emptyset\}$$

que sabemos que no es decidible por teoría.

Dada una MT M , construimos la instancia $(M, U, 1)$, con U es una MT que acepta todas las palabras, es decir, $L(U) = \{0, 1\}^*$ (por ejemplo, U tiene el estado inicial también final). Entonces

$$(M, U, 1) \in L \iff |L(M) \cap L(U)| \geq 1 \iff L(M) \neq \emptyset$$

donde en la última equivalencia se ha usado que $L(M) \cap L(U) = L(M)$, y $|L(M)| \geq 1 \iff L(M) \neq \emptyset$. Tenemos entonces que

$$M \in \text{C-VACIO} \iff (M, U, 1) \in L$$

Así, $\text{C-VACIO} \propto L$, y si L fuera decidible también lo sería C-VACIO, cosa que es falsa, luego L no es decidible.

Ejercicio 1.1.14. Demostrar que los siguientes lenguajes son recursivos:

- El conjunto de las MT M tales que al comenzar con la cinta en blanco, en algún momento escribirán un símbolo no blanco en la cinta.

Sea

$$L_a = \{\langle M \rangle : M \text{ empezando con la cinta en blanco escribe alguna vez un símbolo distinto de } \#\}$$

Teniendo en cuenta que, si la cinta comienza entera en blanco (entonces M se ejecuta sobre la palabra vacía), seguirá estándolo mientras no se escriba ningún símbolo no blanco. Por tanto, en cada paso M lee siempre $\#$. El comportamiento de M se puede determinar entonces por transiciones de la forma $\delta(q, \#)$. Como una MT tiene un conjunto finito de estados y transiciones de la forma $\delta(q, b) = (p, c, M)$, con $M \in \{I, S, D\}$, puede decidirse algorítmicamente en una cantidad finita de pasos, y por tanto en tiempo finito.

Algoritmo 10 MT M_{L_a}

Entrada: Una codificación $\langle M \rangle$ de una MT M

$q \leftarrow q_0$

$V \leftarrow \emptyset$

Mientras $q \notin V$ **hacer**

 Añadir q a V

Si $\delta(q, \#)$ no está definida **entonces**

 Rechazar

Fin Si

 Sea $\delta(q, \#) = (p, c, m)$

Si $c \neq \#$ **entonces**

 Aceptar

Fin Si

$q \leftarrow p$

Fin Mientras

Rechazar

Si aparece una transición que escribe un símbolo $c \neq \#$, M escribe un símbolo no blanco, luego aceptamos. En otro caso, la máquina nunca escribirá un símbolo no blanco. Como los estados son finitos, el algoritmo siempre termina, y decide correctamente.

- b) El conjunto de las MT que nunca se mueven a la izquierda.

Sea

$$L_b = \{\langle M \rangle : M \text{ nunca se mueve a la izquierda}\}$$

Para ver que es decidible, basta con buscar si hay alguna transición con movimiento I que pueda ejecutarse. Como antes de la primera vez la máquina se mueve a la izquierda, todos los movimientos son estáticos o a la derecha, la máquina nunca vuelve a leer una casilla anterior. Por tanto, antes de ese posible primer movimiento a la izquierda, lo único que puede leer es, o bien, símbolos de la entrada, o bien, si los lee todos, blancos $\#$.

Así, podemos construir un grafo finito de configuraciones resumidas, con tripletas (q, t, b) , con $q \in Q$ el estado actual, $t \in \{\text{entrada, blancos}\}$ que indica si aún

estamos leyendo parte de la entrada, o ya hemos terminado con la entrada y hemos llegado a blanco, y $b \in B$ que indica el símbolo leído en la casilla actual.

Algoritmo 11 Decisor para L_b

Entrada: Una codificación $\langle M \rangle$ de una MT M .

Construimos un grafo finito cuyos nodos son ternas (q, t, b) .

Los nodos iniciales son $(q_0, \text{entrada}, a)$, para cada $a \in A$, y $(q_0, \text{blancos}, \#)$.

Para cada nodo (q, t, b) **hacer**

Si $\delta(q, b) = (p, c, I)$ **entonces**

 Marcamos (q, t, b) con $(*)$.

Si no, si $\delta(q, b) = (p, c, S)$ **entonces**

 Añadimos una arista dirigida de (q, t, b) a (p, t, c) .

Si no, si $\delta(q, b) = (p, c, D)$ **entonces**

Si $t = \text{entrada}$ **entonces**

 Añadimos aristas dirigidas a $(p, \text{entrada}, a)$, para cada $a \in A$.

Fin Si

 Añadimos una arista dirigida a $(p, \text{blancos}, \#)$.

Fin Si

Fin Para

Si algún nodo marcado con $(*)$ es alcanzable desde algún nodo inicial, rechazar.

En caso contrario, aceptar.

Veamos que decide correctamente L_b . Si algún nodo marcado es alcanzable, entonces existe una entrada y una ejecución de M que llega a una configuración en la que se ejecuta una transición con movimiento I . Por tanto, M sí se mueve a la izquierda, y debemos rechazar $\langle M \rangle$.

Recíprocamente, si ningún marcado es alcanzable, entonces ninguna transición con movimiento I puede ejecutarse antes del primer movimiento a la izquierda. Pero todo movimiento a la izquierda tendría que ser precisamente el primero en algún instante. Por tanto, M nunca se mueve a la izquierda en ninguna ejecución, y debemos aceptar $\langle M \rangle$.

- c) El conjunto de los pares (M, w) tales que M , al actuar sobre la entrada w , nunca lee una casilla de la cinta más de una vez.

Sea

$$L_c = \{ \langle M, w \rangle : M \text{ actuando sobre } w \text{ nunca lee una casilla de la cinta más de una vez} \}$$

Como estamos hablando de “una” cinta, si el cabezal de dicha cinta cambia de sentido (iba hacia la derecha, y vuelve a la izquierda, o iba hacia la izquierda y vuelve a la derecha), necesariamente vuelve a una casilla ya leída.

Por tanto, mientras no se repita ninguna casilla, el movimiento tiene que ser siempre el mismo, o hacia la derecha, o hacia la izquierda (tampoco puede

pararse).

Algoritmo 12 MT M_{L_c}

Entrada: Una codificación $\langle M, w \rangle$

Simulamos M sobre w .

Guardamos la posición actual del cabezal y el conjunto de casillas ya leídas.

También guardamos la dirección del primer movimiento, si ya se ha producido.

Mientras la simulación no haya terminado **hacer**

Si la casilla actual ya había sido leída **entonces**

 Rechazar

Fin Si

 Marcamos la casilla actual como leída.

Si M se para **entonces**

 Aceptar

Fin Si

 Ejecutamos un paso de M .

Si el movimiento cambia respecto al sentido anterior **entonces**

 Rechazar

Fin Si

Si la cabeza está fuera de la zona inicial de la palabra w , leyendo blancos nuevos, y se repite un estado moviéndose siempre en la misma dirección **entonces**

 Aceptar

Fin Si

Fin Mientras

El algoritmo siempre termina porque:

1. Si M vuelve a leer una casilla, lo detectamos y rechazamos.
2. Si M se para antes de repetir casilla, aceptamos.
3. Si M sigue indefinidamente sin repetir casillas, entonces necesariamente avanza siempre en una dirección leyendo blancos nuevos, y como hay finitos estados, algún estado debe repetirse y entonces aceptamos.

Ejercicio 1.1.15. Indicar si los siguientes problemas son decidibles, semidecidibles o no semidecidibles:

- a) (MANUAL) Determinar si una MT se para para cualquier entrada.

Hecho en el manual, sabemos que es no semidecidible. Se usa una reducción desde C-UNIVERSAL.

- b) Determinar si una MT no se para para ninguna entrada.

El complementario de este problema es semidecidible con una MTND que seleccione no determinísticamente una entrada y simule la MT sobre la palabra. Si alguna para, acepta. Dicho complementario no es decidible porque puede

reducirse desde $\text{PARADA}(M, w)$ (simulamos M sobre w . Si M para, entonces la salida para también. Si M no para, tampoco lo hace la salida por el algoritmo), que no es decidible. Por tanto, el problema será no semidecidible.

- c) Determinar si una MT se para, al menos, para una entrada.

Discutido en el apartado anterior. Es el complementario mencionado (semidecidible pero no decidible).

- d) Determinar si una MT no se para, al menos, para una entrada.

Sabemos que PARADA es semidecidible pero no decidible, luego su complementario es no semidecidible. Podemos reducir desde $\text{C-PARADA}(M, w)$ para ver que este problema no es semidecidible (simulamos M sobre w . Si para, la transformación algorítmica para, y si no para, tampoco para la salida del algoritmo).

Ejercicio 1.1.16. Supongamos que tenemos 4 lenguajes L_1, L_2, L_3, L_4 de tal manera que existe una reducción de L_1 a L_2 , de L_2 a L_3 y de L_4 a L_3 . Para cada una de las siguientes afirmaciones, indicar si para todos los casos son CIERTAS, para todos los casos son FALSAS o si son POSIBLES (en algunos casos son ciertas y, en otros, son falsas).

- a) L_1 es recursivamente enumerable pero no recursivo, y L_3 es recursivo.

FALSA. Si L_3 fuera recursivo, entonces también lo sería L_2 , ya que $L_2 \propto L_3$, y por el mismo argumento, también sería recursivo L_1 , pues $L_1 \propto L_2$, y en particular L_1 sería r.e., lo cual es una contradicción.

- b) L_1 no es recursivo y L_4 no es recursivamente enumerable.

POSIBLE. Consideramos L_u (lenguaje asociado a UNIVERSAL), que es r.e. pero no recursivo, y L_d (lenguaje asociado a DIAGONAL), que no es r.e., luego tampoco recursivo.

Ejemplo: $L_i = L_d \quad i = 1, \dots, 4$. Todas las reducciones se dan por la identidad, y $L_1 = L_d$ no es recursivo, y $L_4 = L_d$ no es r.e.

Contraejemplo: $L_i = L_u \quad i = 1, \dots, 4$. Todas las reducciones se dan por la identidad, pero ahora $L_1 = L_u$ no es recursivo, pero $L_4 = L_u$ sí es r.e.

- c) El complementario de L_1 no es recursivamente enumerable, pero el complementario de L_2 es recursivamente enumerable.

FALSA. Sabemos que $L_1 \propto L_2 \iff \overline{L_1} \propto \overline{L_2}$. Entonces, si $\overline{L_2}$ fuera r.e., también lo sería $\overline{L_1}$. Contradicción.

- d) El complementario de L_2 no es recursivo, pero el complementario de L_3 es recursivo.

FALSA. Igual que en el apartado anterior, $L_2 \times L_3 \iff \overline{L_2} \times \overline{L_3}$. Entonces, si $\overline{L_3}$ fuera recursivo, también lo sería $\overline{L_2}$. Contradicción.

- e) Si L_1 es recursivo, entonces el complementario de L_2 es recursivo.

POSIBLE. Consideramos $L_V = \emptyset$ (lenguaje vacío), que es recursivo, luego r.e., y A^* (todas las palabras del alfabeto $A = \{0, 1\}$), que es recursivo.

Ejemplo: $L_i = L_V \quad i = 1, \dots, 4$. Todas las reducciones se dan por la identidad, y $L_1 = L_V$ es recursivo, y $\overline{L_2} = A^*$ es recursivo.

Contraejemplo: $L_1 = A^*$ y $L_i = L_u \quad i = 2, \dots, 4$. Se reduce $A^* = L_1 \times L_2 = L_u$ enviando toda palabra de L_1 a una palabra fija de L_2 . Las reducciones $L_2 \times L_3$ y $L_4 \times L_3$ se dan por la identidad, pero ahora $L_1 = A^*$ es recursivo, pero $\overline{L_2} = \overline{L_u}$, que no es recursivo (porque está asociado a C-UNIVERSAL, que no es semidecidible, luego $\overline{L_u}$ no es r.e., y tampoco recursivo).

- f) Si L_3 es recursivo, entonces el complementario de L_4 es recursivo.

CIERTA. Si L_3 es recursivo, como $L_4 \times L_3$, L_4 es recursivo, y entonces, por un teorema visto en teoría, $\overline{L_4}$ también.

- g) Si L_3 es recursivamente enumerable, entonces la unión de L_2 y L_4 es recursivamente enumerable.

CIERTA. Si L_3 es r.e., como $L_2 \times L_3$ y $L_4 \times L_3$, se tiene que L_2 y L_4 son r.e., y como la unión de dos r.e. es r.e., entonces la afirmación es cierta.

- h) Si L_3 es recursivamente enumerable, entonces la intersección de L_2 y L_4 es recursivamente enumerable.

CIERTA. Ya sabemos que si L_3 es r.e. entonces L_2 y L_4 son r.e., y como la intersección de dos r.e. también es r.e., la afirmación es cierta.

Ejercicio 1.1.17. Sea el problema de determinar si una máquina de Turing acepta a lo más 100 palabras. Determinar si es decidible, semidecidible o no semidecidible. Justificar la respuesta.

Sea $P(M) = \{\langle M \rangle : |L(M)| \leq 100\}$. Veamos que es no semidecidible. Para ello, consideramos el problema complementario $\overline{P}(M) = \{\langle M \rangle : |L(M)| > 100\}$.

La propiedad “ $|L(M)| > 100$ ” solo depende de $L(M)$, no de la MT M concreta. Por tanto, es una propiedad de los lenguajes r.e.. Además, existe una MT que no la cumple (por ejemplo, una MT sin estados finales), y existe una MT que la cumple

(por ejemplo, una MT con mismo estado inicial y final). Por lo tanto, es no trivial, y entonces, por el Teorema de Rice, $\overline{P}$ es no decidible.

Para la semidecidibilidad, usamos una MTND descrita como sigue:

Algoritmo 13 MTND M_P

Entrada: Una codificación $\langle M \rangle$ de una MT M

Selecciona de forma no determinista 101 palabras $u_1, \dots, u_{101} \in \{0, 1\}^*$.

Si $\exists i, j \in \{1, \dots, 101\} : i \neq j \quad \wedge \quad u_i = u_j$ **entonces**
 M_P rechaza.

Fin Si

Para $i = 1, \dots, 101$ **hacer**

Simulamos M con entrada u_i .

Fin Para

Si todas las simulaciones aceptan **entonces**

M_P acepta.

Fin Si

Tenemos entonces que, si M acepta al menos 101 palabras distintas $u_1, \dots, u_{101}$, existe una rama de cómputo de M_P que precisamente toma $u_1, \dots, u_{101}$, comprueba que son distintas dos a dos, simula $M(u_1), \dots, M(u_{101})$, y como todas aceptan, esa rama acepta.

Por otro lado, si M no acepta al menos 101 palabras distintas, entonces toda rama de cómputo de M_P o bien selecciona dos palabras iguales, en cuyo caso rechaza, o bien selecciona 101 palabras distintas. En este último caso, al menos una de ellas no pertenece a $L(M)$, luego al menos una simulación no aceptará y, consecuentemente, esa rama no aceptará. Por lo tanto, M_P acepta exactamente las codificaciones $\langle M \rangle$ tales que M acepta al menos 101 palabras distintas, de donde $\overline{P}$ es semidecidible.

Como $\overline{P}$ es semidecidible pero no decidible, su complementario $\overline{\overline{P}} = P$ es no semidecidible.

Ejercicio 1.1.18. Determinar cuáles de los siguientes problemas son decidibles, semidecidibles o no semidecidibles:

- a) Saber si una MT para una entrada 0011 no va a usar más de 10 casillas de la cinta.

Veamos que es decidible, para ello, siguiendo el manual, basta ver que “el problema se reduce a realizar un número finito de comprobaciones, cada una de ellas en un número finito de pasos.” Definimos

$$P(M) = \{\langle M \rangle : \text{con entrada 0011, no usa más de 10 casillas}\}$$

Primero, la propiedad “con entrada 0011 no usa más de 10 casillas” no es una propiedad del lenguaje $L(M)$, luego no es una propiedad de los lenguajes r.e., y no puede aplicarse el Teorema de Rice. Para ver que es decidible, consideramos una MT M_P que funciona como sigue:

Algoritmo 14 MT M_P

Entrada: Una codificación $\langle M \rangle$ de una MT M

Simulamos M con entrada 0011, partiendo de su configuración inicial.

En una segunda cinta vamos guardando las casillas distintas visitadas por M y su número (contador).

Si contador > 10 **entonces**

M_P rechaza.

Si M se para y contador ≤ 10 **entonces**

M_P acepta.

Si se repite una configuración y contador ≤ 10 **entonces**

M_P acepta.

Al simular M_P , pueden pasar tres cosas:

1. Que intente usar más de 10 casillas, y entonces M_P debe rechazar.
2. Que pare sin superar 10 casillas, y entonces M_P debe aceptar.
3. Que no pare (cicle), pero entonces, como hay finitas configuraciones posibles, necesariamente se repetirá alguna en tiempo finito, y entonces ciclará siempre sin salir de esas 10 casillas, y entonces M_P debe aceptar.

Por lo tanto, M_P es un algoritmo que resuelve el problema, lo cual lo hace decidable.

b) Dadas dos MT saber si aceptan el mismo lenguaje.

Definimos $\text{IGUAL}(M_1, M_2) = \{(\langle M_1 \rangle, \langle M_2 \rangle) : L(M_1) = L(M_2)\}$. Veamos que no es semidecidible. Podría hacerse directamente, pero primero, para ver que no es decidable, fijamos una MT $M_\emptyset$ tal que

$$L(M_\emptyset) = \emptyset.$$

Consideramos el problema

$$P_{M_\emptyset} = \{\langle M \rangle : L(M) = L(M_\emptyset)\}.$$

Como $L(M_\emptyset) = \emptyset$, entonces

$$L(M) = L(M_\emptyset) \iff L(M) = \emptyset.$$

Por tanto,

$$P_{M_\emptyset} = \{\langle M \rangle : L(M) = \emptyset\}.$$

Si IGUAL fuera decidable, también lo sería $P_{M_\emptyset}$, lo cual contradice el Teorema de Rice, ya que la propiedad “ $L(M) = \emptyset$ ” es una propiedad de los lenguajes r.e. (depende solo de $L(M)$) y además es no trivial: la cumple una MT sin estados finales, pero no la cumple una MT que acepte, por ejemplo, la palabra 0. Por tanto, IGUAL no es decidable.

Veamos que, además, no es semidecidible por medio de una reducción desde

$$\text{VACIO}(M) = \{\langle M \rangle : L(M) = \emptyset\}$$

Fijada una MT M , tal que $L(M) = \emptyset$, es decir, $M \in \text{VACIO}$, definimos la reducción

$$F(\langle N \rangle) = (\langle N \rangle, \langle M \rangle)$$

Intuitivamente, comparar una MT fijada vacía convierte VACIO en IGUAL. Entonces:

1. Si $L(N) = \emptyset$, se tiene que $L(N) = \emptyset = L(M)$, luego $F(\langle N \rangle) \in \text{IGUAL}$.
2. Si $L(N) \neq \emptyset$, se tiene que $L(N) \neq L(M)$, luego $F(\langle N \rangle) \notin \text{IGUAL}$.

Se tiene entonces que $\text{VACIO} \propto \text{IGUAL}$. Como VACIO no es semidecidible, tampoco lo es IGUAL.

Ejercicio 1.1.19. Determinar cuáles de los siguientes problemas son decidibles, semidecidibles o no semidecidibles:

- a) Dada una MT M y una palabra u , saber si la MT acepta la palabra u en un número de pasos menor o igual a $|u|$.

Definimos

$$P(M, u) = \{\langle M, u \rangle : M \text{ acepta } u \text{ en un número de pasos menor o igual a } |u|\}.$$

Veamos que es decidible. Para ello, consideramos una MT M_P que funciona como sigue, donde la configuración inicial de una MT con entrada u es (q_0, ε, u) :

Algoritmo 15 MT M_P

Entrada: Una codificación $\langle M, u \rangle$ de una MT M y una palabra u

Para $i = 0, \dots, |u|$ **hacer**

Simulamos i pasos de M con entrada u , partiendo de su configuración inicial.

Si M entra en un estado de aceptación **entonces**

M_P acepta.

Si M para sin aceptar **entonces**

M_P rechaza.

Si $i = |u|$ y M no acepta **entonces**

M_P rechaza.

Al simular M_P , pueden pasar tres cosas:

1. Que M acepte en k pasos con $k \leq |u|$, y entonces M_P debe aceptar.
2. Que M se pare antes o en $|u|$ pasos, pero sin aceptar, y entonces M_P debe rechazar.
3. Que M no haya aceptado tras $|u|$ pasos. En ese caso, aunque después aceptara o ciclara, ya no cumpliría la condición pedida, y entonces M_P debe rechazar.

Por tanto, M_P siempre termina, ya que sólo necesita simular, como mucho, $|u|$ pasos de cálculo, y decide correctamente si M acepta u en un número de pasos menor o igual a $|u|$, y el problema es decidible.

- b) Dada una MT determinar si no acepta la palabra vacía.

Deberíamos intuir, al ser de la forma “determinar si no acepta (algo)” que no es semidecidible. Definimos $P_\varepsilon = \{\langle M \rangle : M \text{ no acepta } \varepsilon\}$. Vemos que es el complementario de $C - P_\varepsilon = \{\langle M \rangle : M \text{ acepta } \varepsilon\}$. Sin embargo, $C - P_\varepsilon$ es semidecidible, ya que podemos construir una MT M_{P_ε} que, con entrada $\langle M \rangle$, simule a M sobre la palabra vacía ε , y acepte si M acepta. Si además P_ε fuera semidecidible entonces P_ε y $C - P_\varepsilon$ serían semidecidibles, lo que haría que P_ε fuera decidible, y a su vez, que $C - P_\varepsilon$, al ser complementario de decidible, también lo fuera. Sin embargo, $C - P_\varepsilon$ no es decidible, ya que “aceptar ε ” depende solo de $L(M)$, porque equivale a ver si $\varepsilon \in L(M)$. Entonces es una propiedad de los lenguajes r.e., y además es no trivial, ya que hay una MT cuyo lenguaje aceptado es $\{\varepsilon\}$ (por ejemplo, una transición que lea blanco y pase a estado final), que sí la cumple, pero también otra MT cuyo lenguaje aceptado es $\{0\}$, que no la cumple. Por tanto, $C - P_\varepsilon$ no es decidible por el Teorema de Rice, lo cual sería una contradicción. Necesariamente entonces P_ε no es semidecidible.

Ejercicio 1.1.20. Determinar cuáles de los siguientes problemas son decidibles, semidecidibles o no semidecidibles (se supone que las MTs tienen a $A = \{0, 1\}$ como alfabeto de entrada):

- a) Dadas dos máquinas de Turing, M_1 y M_2 , determinar si existe, al menos, una palabra aceptada simultáneamente por ambas máquinas.

Sea $P = \{\langle M_1, M_2 \rangle : L(M_1) \cap L(M_2) \neq \emptyset\}$. Veamos que es semidecidible pero no decidible. La no decidibilidad se deduce del Teorema de Rice, ya que, fijada una MT M_* que acepte todas las palabras, $L(M_*) = A^*$, consideramos el problema siguiente

$$P_{M_*} = \{\langle M \rangle : L(M) \cap L(M_*) \neq \emptyset\}$$

Como $L(M_*) = A^*$, entonces

$$L(M) \cap L(M_*) \neq \emptyset \iff L(M) \neq \emptyset$$

Por tanto, $P_{M_*} = \{\langle M \rangle : L(M) \neq \emptyset\}$. Si P fuera decidible, también lo sería P_{M_*} , lo cual es una contradicción con el Teorema de Rice, ya que la propiedad “ $L(M) \neq \emptyset$ ” es una propiedad de los lenguajes r.e. (solo depende de $L(M)$) y además es no trivial, ya que si M es una MT sin estados finales, entonces $L(M) = \emptyset$, y si M es una MT que acepta todas las palabras, se tiene que $L(M) = A^* \neq \emptyset$. Por tanto, P es no decidible.

La semidecidibilidad se obtiene con la siguiente MTND:

Algoritmo 16 MTND M_P

Entrada: Un par de codificaciones $\langle M_1, M_2 \rangle$ de dos MT M_1 y M_2

Selecciona de forma no determinista una palabra $w \in A^*$.

Para $i = 1, 2$ **hacer**

 Simulamos M_i con entrada w .

Fin Para

Si ambas simulaciones aceptan **entonces**

M_P acepta.

Fin Si

- b) (MANUAL) Dada una MT, determinar si para toda palabra de entrada no realiza más de 5 movimientos.

Hecho en el manual con 10 movimientos (pasos de cálculo). La clave está en ver que en 5 movimientos solo se pueden leer cinco casillas de la cinta como máximo. Entonces las palabras se identifican para este problema con los prefijos de longitud, a lo sumo, 5. Así, dada una palabra $u \in A^*$ tal que $|u| \leq 5$, y dada una palabra $v \in A^*$, entonces, para una MT M cualquiera,

M no realiza más de 5 movimientos con $u \iff M$ no realiza más de 5 movimientos con uv

El conjunto $\{u \in A^* : |u| \leq 5\}$ es finito. De hecho, podemos obtener su cardinal:

$$\sum_{k=0}^5 2^k = \frac{2^6 - 1}{2 - 1} = 63$$

y basta ejecutar 5 pasos de cálculo para cada una de estas 63 palabras. Si con todas para, la respuesta es que sí, y si con alguna no ha parado, la respuesta es que no.

- c) Determinar si una MT no es determinista.

Sea $P = \{\langle M \rangle : M \text{ es una MTND}\}$. Podemos utilizar el siguiente algoritmo para resolver el problema:

Como Q , B y la tabla de transiciones de M son finitos, el algoritmo siempre termina, y el algoritmo anterior responde correctamente en todos los casos. Por tanto, P es decidable y, en particular, semidecidible.

- d) (MANUAL) Dada una MT, determinar si ninguna de las palabras que acepta es un palíndromo.

Es el complementario del problema del Ejercicio 1.1.9, que es semidecidible pero no decidable, luego este problema es no semidecidible.

Algoritmo 17 MT M_P

Entrada: Una codificación $\langle M \rangle$ de una MT M

Leemos la codificación de M , en particular su conjunto de estados Q , su alfabeto de cinta B y su tabla de transiciones.

Para cada par $(q, a) \in Q \times B$ **hacer**

Contamos cuántas transiciones están definidas desde (q, a) .

Si hay dos o más transiciones definidas desde (q, a) **entonces**

M_P acepta.

Fin Si

Fin Para

M_P rechaza.

Ejercicio 1.1.21. Se considera el siguiente problema: dadas dos máquinas de Turing, determinar si aceptan el mismo lenguaje. Razonar si este problema es decidible, semidecidible o no semidecidible.

Es igual que el Ejercicio 1.1.18 b).

Ejercicio 1.1.22. Determinar, justificando las respuestas, cuáles de los siguientes problemas son decidibles, semidecidibles o no semidecidibles:

- a) Dada una máquina de Turing y una palabra de entrada determinar si visita menos de 10 casillas distintas de la cinta de lectura.

Es igual que el Ejercicio 1.1.18 a), pero con una entrada $w \in \{0, 1\}^*$ cualquiera.

Solo cambiaría la definición del problema

$$P(M, w) = \{\langle M, w \rangle : M \text{ con entrada } w \text{ visita menos de 10 casillas distintas}\}$$

y la entrada del algoritmo sería una codificación $\langle M, w \rangle$ de una MT M y una palabra w , y habría que simular M sobre w y seguir la misma lógica que en dicho ejercicio.

Este problema también es decidible.

- b) Dadas dos Máquinas de Turing, determinar si el conjunto de las palabras aceptadas a la vez por ambas máquinas de Turing es finito.

Podríamos ver en dos pasos no decidibilidad con Rice, y luego no semidecidibilidad, pero veamos directamente que no es semidecidible por medio de una reducción desde VACIO (que sabemos por teoría que no es semidecidible).

Definimos $P(M_1, M_2) = \{\langle M_1, M_2 \rangle : L(M_1) \cap L(M_2) \text{ es finito}\}$. Reducimos desde

$$\text{VACIO}(M) = \{\langle M \rangle : L(M) = \emptyset\}$$

Algoritmo 18 MT M' **Entrada:** Una palabra $x \in \{0, 1\}^*$ Ignoramos la entrada x .**Para** $t = 1, \dots, \infty$ **hacer** **Para** cada palabra $w \in \{0, 1\}^*$ con $|w| \leq t$ **hacer** Simulamos M con entrada w durante t pasos. **Si** M acepta w en esos t pasos **entonces** M' acepta x . **Fin Si** **Fin Para****Fin Para**Dada una MT M , construimos M' dada como sigue:

Es decir, se tiene que

$$L(M') = \begin{cases} \emptyset & \text{si } L(M) = \emptyset \\ \{0, 1\}^* & \text{si } L(M) \neq \emptyset \end{cases}$$

Fijamos ahora una MT M_0 tal que $L(M_0) = \{0, 1\}^*$. Definimos la reducción

$$F(\langle M \rangle) = (\langle M' \rangle, \langle M_0 \rangle)$$

Entonces $L(M') \cap L(M_0) = L(M')$, y

$$L(M) = \emptyset \iff L(M') = \emptyset \iff L(M') \cap L(M_0) \text{ es finito}$$

Por tanto, $\text{VACIO} \propto P$. Como VACIO no es semidecidible, tampoco lo es P .**Ejercicio 1.1.23.** Indica cuáles de los siguientes problemas son decidibles, semidecidibles o no semidecidibles:

- a) Determinar si el lenguaje aceptado por una MT es regular.

Sea $\text{REGULAR} = \{\langle M \rangle : L(M) \text{ es regular}\}$. Podemos ver que no es decidible y luego que no es semidecidible en dos pasos, o bien directamente que no es semidecidible por medio de una reducción desde VACIO . Para ver que es no decidible usamos el Teorema de Rice, ya que la propiedad “ $L(M)$ es regular” depende solo de $L(M)$, luego es una propiedad de los lenguajes r.e. y es no trivial, porque existe una MT M_1 con $L(M_1) = \emptyset$ (sin estados finales por ejemplo), y $L(M_1)$ cumple la propiedad, y existe una MT M_2 con $L(M_2) = \{0^n 1^n : n \geq 0\}$ (por ejemplo, multicinta que cuente cuántos 0’s aparecen y ponga ese mismo número de 1’s a continuación), y $L(M_2)$ no cumple la propiedad.

La no semidecidibilidad puede obtenerse con una reducción desde

$$\text{VACIO}(M) = \{\langle M \rangle : L(M) = \emptyset\}$$

Algoritmo 19 MT M'

Entrada: Una palabra $x \in \{0, 1\}^*$
 Comprobamos si $x \in \{0^n 1^n : n \geq 0\}$.
Si $x \notin \{0^n 1^n : n \geq 0\}$ **entonces**
 M' rechaza.
Fin Si
Para $t = 1, \dots, \infty$ **hacer**
 Para cada palabra $w \in \{0, 1\}^*$ con $|w| \leq t$ **hacer**
 Simulamos M con entrada w durante t pasos.
 Si M acepta w en esos t pasos **entonces**
 M' acepta x .
 Fin Si
 Fin Para
Fin Para

Dada una MT M , construimos una MT M' con el siguiente algoritmo:

Tendremos entonces que

$$L(M') = \begin{cases} \emptyset & \text{si } L(M) = \emptyset \\ \{0^n 1^n : n \geq 0\} & \text{si } L(M) \neq \emptyset \end{cases}$$

Como $\emptyset$ es regular, pero $\{0^n 1^n : n \geq 0\}$ no lo es, se tiene que

$$L(M) = \emptyset \iff L(M') = \emptyset \iff L(M') \text{ es regular}$$

Definimos la reducción $F(\langle M \rangle) = \langle M' \rangle$. Viendo que

$$\langle M \rangle \in \text{VACIO} \iff F(\langle M \rangle) = \langle M' \rangle \in \text{REGULAR}$$

deducimos que $\text{VACIO} \propto \text{REGULAR}$. Como VACIO no es semidecidible, tampoco lo es REGULAR.

- b) Dadas dos MTs determinar si hay una palabra que es aceptada por las dos MT y además es un palíndromo.

Sea $P = \{\langle M_1, M_2 \rangle : \exists w \in \{0, 1\}^* \text{ palíndromo} : w \in L(M_1) \cap L(M_2)\}$. Veamos que es semidecidible pero no decidible. La no decidibilidad se deduce del Teorema de Rice, ya que, fijada una MT M_* que acepte todas las palabras, $L(M_*) = A^*$, consideramos el problema siguiente

$$P_{M_*} = \{\langle M \rangle : \exists w \in \{0, 1\}^* \text{ palíndromo} : w \in L(M) \cap L(M_*)\}$$

Como $L(M_*) = A^*$, entonces, considerando $\text{PAL} = \{w \in \{0, 1\}^* : w \text{ es un palíndromo}\}$:

$$\exists w \in \{0, 1\}^* \text{ palíndromo} : w \in L(M) \cap L(M_*) \iff L(M) \cap \text{PAL} \neq \emptyset$$

Por tanto, $P_{M_*} = \{\langle M \rangle : L(M) \cap \text{PAL} \neq \emptyset\}$. Si P fuera decidible, también lo sería P_{M_*} , lo cual es una contradicción con el Teorema de Rice, ya que la

propiedad “ $L(M) \cap \text{PAL} \neq \emptyset$ ” es una propiedad de los lenguajes r.e. (solo depende de $L(M)$) y además es no trivial, ya que si M es una MT sin estados finales, entonces $L(M) = \emptyset$, y no cumple la propiedad, y si M es una MT que acepta todas las palabras, se tiene que $L(M) = A^* \neq \emptyset$, que sí cumple la propiedad (contiene a todos los palíndromos de hecho). Por tanto, P es no decidible.

La semidecidibilidad se obtiene con la siguiente MTND:

Algoritmo 20 MTND M_P

Entrada: Un par de codificaciones $\langle M_1, M_2 \rangle$ de dos MT M_1 y M_2

Selecciona de forma no determinista una palabra $w \in A^*$.

Comprueba si w es un palíndromo.

Si w no es un palíndromo **entonces**

M_P rechaza.

Fin Si

Para $i = 1, 2$ **hacer**

Simulamos M_i con entrada w .

Fin Para

Si ambas simulaciones aceptan **entonces**

M_P acepta.

Fin Si

Ejercicio 1.1.24. Indica cuáles de los siguientes problemas son decidibles, semidecidibles o no semidecidibles:

- a) Dada una MT determinar si todos sus estados son accesibles (se puede llegar a ellos para un cálculo para alguna entrada).

Sea

$$P = \{ \langle M \rangle : \forall q \in Q_M, \exists w \in \{0, 1\}^* : \exists t \in \mathbb{N} : M(w) \text{ accede al estado } q \text{ en } t \text{ pasos} \}$$

El problema es semidecidible pero no decidible. Podemos ver la semidecidibilidad con la siguiente MTND:

Si todos los estados son accesibles, existirá, para cada $i = 1, \dots, k$, un estado $q_i \in Q_M$, una palabra $w_i \in \{0, 1\}^*$ y un número t_i tales que la ejecución de M sobre w_i visita q_i en, a lo sumo, t_i pasos.

Para probar que P no es decidible, reducimos desde

$$\text{UNIVERSAL}(M, w) = \{ \langle M, w \rangle : M \text{ acepta } w \},$$

que sabemos que no es decidible.

Dada una instancia $\langle M, w \rangle$ de UNIVERSAL, construimos una MT M' tal que todos sus estados sean accesibles salvo, quizá, un estado especial q_* . La máquina M' ignora su entrada y funciona como sigue:

Algoritmo 21 MTND M_P

Entrada: Una codificación $\langle M \rangle$ de una MT M

Sea $Q_M = \{q_1, \dots, q_k\}$ el conjunto de estados de M .

Para $i = 1, \dots, k$ **hacer**

 Selecciona de forma no determinista una palabra $w_i \in \{0, 1\}^*$.

 Selecciona de forma no determinista un número $t_i \in \mathbb{N}$.

Fin Para

Para $i = 1, \dots, k$ **hacer**

 Simula M con entrada w_i durante, a lo sumo, t_i pasos.

Si durante esa simulación no se visita el estado q_i **entonces**

M_P rechaza.

Fin Si

Fin Para

M_P acepta.

Algoritmo 22 MT M'

Entrada: Una palabra $x \in \{0, 1\}^*$

M' ignora la entrada x .

Obtenemos $Q' \setminus \{q_*\} = \{p_0, p_1, \dots, p_m\}$, donde p_0 es el estado inicial de M' .

M' realiza una subrutina de accesibilidad que recorre todos los estados de M' salvo q_* , mediante transiciones auxiliares que no interfieren con la simulación posterior.

Al llegar al estado p_m , M' coloca en la cinta $\langle M \rangle 111w$.

Desde p_m , M' comienza la simulación de M con entrada w .

Si M acepta w **entonces**

M' entra en el estado especial q_* y acepta.

Fin Si

Si M rechaza w **entonces**

M' rechaza sin pasar por q_* .

Fin Si

Si M no para con w , entonces M' tampoco para y nunca entra en q_* .

Por construcción, todos los estados de M' distintos de q_* son accesibles, ya que la subrutina de accesibilidad pasa por todos ellos.

El único estado cuya accesibilidad no queda garantizada entonces es q_* . Además, por construcción, M' entra en q_* si y solo si la simulación de M sobre w acepta. Deducimos de lo anterior que

$$q_* \text{ es accesible en } M' \iff M \text{ acepta } w.$$

Como todos los demás estados son accesibles por construcción, se tiene

$$\text{todos los estados de } M' \text{ son accesibles} \iff q_* \text{ es accesible en } M' \iff M \text{ acepta } w.$$

Por tanto,

$$\langle M, w \rangle \in \text{UNIVERSAL} \iff \langle M' \rangle \in P.$$

Así,

$$\text{UNIVERSAL} \propto P.$$

Si P fuera decidable, entonces UNIVERSAL también sería decidable, contradicción. Por tanto, P no es decidable.

b) Dada una MT determinar si su número de estados es menor o igual a 5.

Sea $P = \{\langle M \rangle : |Q_M| \leq 5\}$. El problema es decidable, y se resuelve con el siguiente algoritmo:

Algoritmo 23 MT M_P

Entrada: Una codificación $\langle M \rangle$ de una MT M

Comprobamos que $\langle M \rangle$ es una codificación correcta de una MT.

Leemos el conjunto de estados Q de M .

Contamos cuántos estados tiene M .

Si $|Q| \leq 5$ **entonces**

M_P acepta.

Si no

M_P rechaza.

Fin Si

Ejercicio 1.1.25. Demuestra que el Problema de la Correspondencia de Post con un alfabeto A que tiene un sólo elemento es decidable.

Sea $a \in A$ tal que $A = \{a\}$. Entonces, $A^* = \{a^n : n \geq 0\}$. Cada bloque del PCP será de la forma

$$\frac{a^{p_i}}{a^{q_i}}$$

con p_i la longitud de la palabra superior, y q_i la longitud de la palabra inferior, $i = 1, \dots, k$, y k el número de bloques. Entonces, para una sucesión no vacía de enteros

$i_1, \dots, i_m$, la palabra de arriba será $a^{p_{i_1}} \dots a^{p_{i_m}} = a^{p_{i_1} + \dots + p_{i_m}}$. Análogamente, la palabra de abajo será $a^{q_{i_1} + \dots + q_{i_m}}$. Entonces, como solo hay un símbolo, dos palabras son iguales si y solo si tienen la misma longitud, es decir, la sucesión es válida (como solución al PCP) si y solo si $p_{i_1} + \dots + p_{i_m} = q_{i_1} + \dots + q_{i_m}$. Definimos $d_i = p_i - q_i$. La condición de solución se reescribe como

$$p_{i_1} + \dots + p_{i_m} = q_{i_1} + \dots + q_{i_m} \iff (p_{i_1} - q_{i_1}) + \dots + (p_{i_m} - q_{i_m}) = 0 \iff d_{i_1} + \dots + d_{i_m} = 0$$

Para ver que es decidible usamos el siguiente algoritmo:

Algoritmo 24 MT M_{PCP}

Entrada: Una instancia de PCP sobre $A = \{a\}$ con bloques $\frac{a^{p_1}}{a^{q_1}}, \dots, \frac{a^{p_k}}{a^{q_k}}$.

Para $i = 1, \dots, k$ **hacer**

Calculamos $d_i = p_i - q_i$.

Fin Para

Si existe i tal que $d_i = 0$ **entonces**

Aceptamos.

Fin Si

Si existen i, j tales que $d_i > 0$ y $d_j < 0$ **entonces**

Aceptamos.

Fin Si

Rechazamos.

Si algún $d_i = 0$, entonces el bloque i es solución. Si hay $d_i > 0$ y $d_j < 0$ para $i, j \in \{1, \dots, k\}$, escribimos $d_i = r > 0$, y $d_j = -s < 0$. Tomando s copias del bloque i y r copias del bloque j , la diferencia total será $s \cdot r + r \cdot (-s) = 0$, luego hay solución. Si todos los d_i son positivos, cualquier suma no vacía será positiva. Si, por otro lado, todos los d_i son negativos, cualquier suma no vacía será negativa. En ninguno de estos dos casos puede haber solución. Como el algoritmo solo recorre un número finito de bloques y siempre termina, el PCP sobre un alfabeto con un único símbolo es decidible.

2. Ejercicios sobre Clases de Complejidad

Ejercicio 1.

- (a) Un grafo dirigido se dice que es *acíclico* si no tiene ciclos. Demostrar que todo grafo finito dirigido acíclico tiene una *fuentes* (un nodo al que no llegan arcos).
- (b) Utilizar (a) para demostrar que un grafo dirigido con n nodos es acíclico si y solo si se pueden numerar los nodos del 1 al n de manera que siempre los arcos van desde números más pequeños a números más grandes.
- (c) Utilizar (a) y (b) para describir un algoritmo en tiempo polinómico para determinar cuándo un grafo dirigido es acíclico.

Ejercicio 2.

- (a) Demostrar que un grafo es *bipartito* (se puede dividir en dos partes, que no son necesariamente iguales, de manera que todas las aristas van de una parte a otra) si y solo si todos sus ciclos son de longitud par.
- (b) Utilizar (a) para describir un algoritmo en tiempo polinómico para comprobar si un grafo es bipartito.

Ejercicio 3. Demostrar que $\mathbf{P}$ es cerrada para la unión y la intersección.

Ejercicio 4. Sea C una clase de funciones de los enteros no negativos en los enteros no negativos. Se dice que C es cerrada para la composición polinómica por la izquierda si $f(n) \in C$ implica que $p(f(n)) = O(g(n))$ para alguna función $g(n) \in C$, donde $p(n)$ es un polinomio cualquiera. Se dice que C es cerrada para la composición polinómica por la derecha si $f(n) \in C$ implica que $f(p(n)) = O(g(n))$ para alguna función $g(n) \in C$, donde $p(n)$ es un polinomio cualquiera.

Determinar cuáles de las siguientes clases de funciones son cerradas para la composición polinómica por la derecha y cuáles lo son por la izquierda.

- (a) $\{n^k : k > 0\}$.
- (b) $\{n \cdot k : k > 0\}$.
- (c) $\{k^n : k > 0\}$.
- (d) $\{2^{n^k} : k > 0\}$.

(e) $\{\log^k(n) : k > 0\}$.

(f) $\{\log(n)\}$.

Nota: en los dos últimos casos se entiende que las funciones se redondean al entero más cercano. En vez de considerar todos los polinomios, como estamos hablando de órdenes de complejidad, será suficiente considerar los polinomios de la forma $p(n) = n^l$, donde l es un número natural.

Ejercicio 5. Sea $L = \{wcw : w \in \{0,1\}^*\}$. Demostrar que L se puede reconocer en espacio $\log(n)$.

Ejercicio 6. Demostrar que si L está en $\mathbf{P}$, entonces L^* también está en $\mathbf{P}$.

Ejercicio 7. Demostrar que si L está en $\mathbf{NP}$, entonces L^* también está en $\mathbf{NP}$.

Ejercicio 8. Demostrar que $\mathbf{NP}$ es cerrada para la unión y la intersección.

Ejercicio 9. Demostrar que si $\mathbf{NP} \neq \mathbf{CoNP}$, entonces $\mathbf{P} \neq \mathbf{NP}$.

Ejercicio 10. Supongamos que una entrada es una palabra de paréntesis. Demostrar que determinar si están emparejados y anidados correctamente está en $\mathbf{L}$. Lo están $((()))$ y $((()))$, pero no lo están $)()()$ ni $((()))()$.

Ejercicio 11. Consideremos el problema anterior, pero ahora con dos tipos de paréntesis: $(,)$ y $[,]$. Demostrar que determinar si están bien escritos también está en $\mathbf{L}$. Aquí $([]())$ está permitido, pero $([])$ no. Demostrar que si la entrada se lee de izquierda a derecha sin volver hacia atrás, requieren $O(n)$ de memoria.

Ejercicio 12. Demostrar que el problema del palíndromo requiere $O(n)$ de memoria si no se puede volver hacia atrás.

Ejercicio 13. Demostrar que $\mathbf{NP} \neq \mathbf{ESPACIO}(n)$. Nota: Este ejercicio es muy complicado. Como sugerencia, para probar que las clases son diferentes, probar que $\mathbf{NP}$ cumple la propiedad de ser cerrada para reducciones en L (si $P_1 \propto P_2$ y P_2 está en $\mathbf{NP}$, entonces P_1 está en $\mathbf{NP}$), mientras que $\mathbf{ESPACIO}(n)$, no.

2.1. Soluciones

Ejercicio 2.1.1.

- (a) Un grafo dirigido se dice que es *acíclico* si no tiene ciclos. Demostrar que todo grafo finito dirigido acíclico tiene una *fuentes* (un nodo al que no llegan arcos).

Sea $G = (V, E)$ un grafo dirigido finito acíclico. Por ser G finito, podemos suponer que $|V| = n \in \mathbb{N}$. Supongamos ahora, por reducción al absurdo, que G no tiene ninguna fuente. Entonces, todo nodo de G tiene al menos un arco incidente. Tomamos un nodo cualquiera $v_1 \in V$. Como v_1 no es fuente, existe un nodo $v_2 \in V$ tal que $(v_2, v_1) \in E$. De nuevo, como v_2 no es fuente, existe $v_3 \in V$ tal que $(v_3, v_2) \in E$. Repitiendo el argumento, obtenemos una sucesión de nodos

$$v_1, v_2, \dots, v_{n+1}$$

tales que, para todo $i = 1, \dots, n$,

$$(v_{i+1}, v_i) \in E.$$

Como estos $n + 1$ nodos pertenecen al conjunto de vértices V , que tiene solamente n elementos, existen $1 \leq i < j \leq n + 1$ tales que $v_i = v_j$. Entonces tenemos el ciclo dirigido

$$v_j \rightarrow v_{j-1} \rightarrow \dots \rightarrow v_i = v_j,$$

lo cual contradice que G sea acíclico. Por tanto, G tiene al menos una fuente.

- (b) Utilizar (a) para demostrar que un grafo dirigido con n nodos es acíclico si y solo si se pueden numerar los nodos del 1 al n de manera que siempre los arcos van desde números más pequeños a números más grandes.

Queremos demostrar que, llamando P a la propiedad de numeración del enunciado, si G es un grafo dirigido con n nodos, entonces

$$G \text{ es acíclico} \iff \text{se cumple } P$$

$\implies$) Supongamos primero que G es acíclico. Como G es finito y tiene n nodos, por el apartado (a), G tiene una fuente. Numeramos dicha fuente con el número 1 y la eliminamos del grafo junto con todos sus arcos salientes. El grafo restante sigue siendo finito y acíclico, por ser este subgrafo de un grafo finito y acíclico. Por tanto, si queda algún nodo, podemos volver a aplicar el apartado (a) y obtener una nueva fuente. La numeramos con el número 2 y la eliminamos junto con sus arcos salientes. Repitiendo este proceso, como inicialmente había n nodos, tras n pasos todos los nodos quedan numerados con los números $1, \dots, n$. Veamos que esta numeración cumple la propiedad pedida. Sea $(u, v) \in E$ un arco cualquiera del grafo original. Supongamos que v se numeró antes que u . Entonces, en el momento en que se numeró v , el nodo u todavía no había sido eliminado.

Además, el arco (u, v) tampoco había sido eliminado, pues solo se eliminan los arcos salientes de los nodos que se van numerando, y u todavía seguía en el grafo. Por tanto, en ese momento v tenía un arco entrante, luego no podía ser fuente, pero si se había eliminado v era porque era fuente, lo cual es una contradicción. Así, necesariamente u se numera antes que v , es decir, $\text{num}(u) < \text{num}(v)$ y se cumple P .

($\Leftarrow$) Recíprocamente, supongamos que se cumple P , es decir, todos los nodos de G se pueden numerar del 1 al n de forma que todo arco va desde un nodo con número menor a un nodo con número mayor. Veamos que G es acíclico. Si G tuviera un ciclo dirigido

$$v_1 \rightarrow v_2 \rightarrow \cdots \rightarrow v_k \rightarrow v_1,$$

para cierto $1 \leq k \leq n$ entonces, como por hipótesis se cumple P , tendríamos

$$\text{num}(v_1) < \text{num}(v_2) < \cdots < \text{num}(v_k) < \text{num}(v_1) \implies \text{num}(v_1) < \text{num}(v_1)$$

lo cual contradice que se cumpla P . Por tanto, G no tiene ciclos dirigidos, es decir, G es acíclico.

(c) Utilizar (a) y (b) para describir un algoritmo en tiempo polinómico para determinar cuándo un grafo dirigido es acíclico.

El algoritmo es el siguiente:

Algoritmo 25 Decidir si un grafo dirigido es acíclico

Entrada: Un grafo dirigido finito $G = (V, E)$

Mientras $V \neq \emptyset$ **hacer**

 Buscar un nodo $v \in V$ fuente.

Si no existe tal nodo **entonces**

 Rechazar, es decir, responder que G no es acíclico.

Fin Si

 Eliminar v del grafo, junto con todos sus arcos salientes.

Fin Mientras

Aceptar, es decir, responder que G es acíclico.

Veamos que el algoritmo es correcto. Si el algoritmo acepta, entonces ha eliminado todos los nodos del grafo. Por tanto, podemos numerar los nodos en el mismo orden en que han sido eliminados. Como en cada paso se elimina un nodo fuente, por el razonamiento del apartado (b), todo arco del grafo original va desde un nodo eliminado antes hacia un nodo eliminado después. Así, se obtiene una numeración de los nodos del 1 al n en la que todos los arcos van desde números más pequeños a números más grandes. Nuevamente por el apartado (b), G es acíclico.

Si el algoritmo rechaza, entonces en algún momento queda un subgrafo no vacío sin ninguna fuente. Por el apartado (a), dicho subgrafo no puede ser

acíclico, ya que todo grafo dirigido finito acíclico tiene una fuente. Por tanto, ese subgrafo tiene un ciclo dirigido. Como dicho subgrafo procede de eliminar nodos y arcos de G , ese ciclo también estaba en G . Luego G no es acíclico.

Finalmente, el algoritmo tiene tiempo polinómico. En cada iteración se puede buscar una fuente recorriendo los nodos y los arcos del grafo restante. Como en cada iteración se elimina un nodo, hay a lo sumo n iteraciones. Por tanto, el tiempo total está acotado por un polinomio en el tamaño de la entrada.

Ejercicio 2.1.2.

- (a) Demostrar que un grafo es *bipartito* (se puede dividir en dos partes, que no son necesariamente iguales, de manera que todas las aristas van de una parte a otra) si y solo si todos sus ciclos son de longitud par.

Sea G un grafo. Queremos demostrar que, llamando P a la propiedad de que todos los ciclos de G tienen longitud par, entonces

$$G \text{ es bipartito} \iff \text{se cumple } P.$$

$\implies$) Supongamos primero que $G = (V, E)$ es bipartito. Entonces existen dos subconjuntos $V_1, V_2 \subset V$ tales que

$$V = V_1 \cup V_2, \quad V_1 \cap V_2 = \emptyset,$$

y toda arista de G tiene un extremo origen en V_1 y el otro destino en V_2 . Sea

$$v_1 \rightarrow v_2 \rightarrow \cdots \rightarrow v_k \rightarrow v_1$$

un ciclo de G , para cierto $k \in \mathbb{N}$. Sin pérdida de generalidad, supongamos que $v_1 \in V_1$. Como toda arista va de una parte a la otra, entonces $v_2 \in V_2$, $v_3 \in V_1$, $v_4 \in V_2$, y así sucesivamente. Por tanto, los vértices en posiciones impares pertenecen a V_1 y los vértices en posiciones pares pertenecen a V_2 . Como el último arco del ciclo es (v_k, v_1) , el vértice v_k debe estar en una parte distinta de la de v_1 . Como $v_1 \in V_1$, necesariamente $v_k \in V_2$. Por lo anterior, esto ocurre si y solo si k es par. Luego todo ciclo de G tiene longitud par.

($\impliedby$) Recíprocamente, supongamos que todos los ciclos de G tienen longitud par. Veamos que G es bipartito. Podemos suponer primero que G es conexo. Tomamos un vértice cualquiera $s \in V$ y definimos

$$V_1 = \{v \in V : \text{existe un camino de longitud par desde } s \text{ hasta } v\},$$

$$V_2 = \{v \in V : \text{existe un camino de longitud impar desde } s \text{ hasta } v\}.$$

Veamos que esta definición es correcta, es decir, que ningún vértice puede pertenecer a la vez a V_1 y a V_2 . Supongamos, por reducción al absurdo, que existe un vértice $v \in V_1 \cap V_2$. Entonces existe un camino de longitud par desde s hasta v y también un camino de longitud impar desde s hasta v . Tomando ambos caminos, y eliminando las partes comunes si

fuera necesario, se obtiene un ciclo de longitud impar. Esto contradice la hipótesis de que todos los ciclos de G tienen longitud par. Por tanto, $V_1 \cap V_2 = \emptyset$.

Como G es conexo, todo vértice es alcanzable desde s , luego todo vértice pertenece a V_1 o a V_2 . Así,

$$V = V_1 \cup V_2.$$

Veamos ahora que toda arista tiene un extremo en V_1 y el otro en V_2 . Sea $(u, v) \in E$. Si u y v pertenecieran al mismo conjunto (V_1 o V_2), entonces existirían caminos desde s hasta u y desde s hasta v de la misma paridad. Al añadir la arista (u, v) , se obtendría un ciclo de longitud impar, lo cual es una contradicción con la hipótesis de que todos los ciclos tienen longitud par. Por tanto, toda arista une un vértice de V_1 con un vértice de V_2 . Luego G es bipartito. Si G no es conexo, aplicamos el mismo razonamiento a cada componente conexa y unimos las dos partes obtenidas en cada componente. Por tanto, también en el caso general G es bipartito.

- (b) Utilizar (a) para describir un algoritmo en tiempo polinómico para comprobar si un grafo es bipartito.

El algoritmo es el siguiente:

Algoritmo 26 Decidir si un grafo es bipartito

Entrada: Un grafo $G = (V, E)$

Inicialmente, todos los vértices están sin colorear.

Para cada vértice $s \in V$ sin colorear **hacer**

Colorear s con el color 1.

Inicializar una cola con el vértice s .

Mientras la cola no esté vacía **hacer**

Extraer un vértice u de la cola.

Para cada vecino v de u **hacer**

Si v no está coloreado **entonces**

Colorear v con el color distinto al de u .

Añadir v a la cola.

Si no, si v tiene el mismo color que u **entonces**

Rechazar, es decir, responder que G no es bipartito.

Fin Si

Fin Para

Fin Mientras

Fin Para

Aceptar, es decir, responder que G es bipartito.

Veamos que el algoritmo es correcto. Si el algoritmo acepta, entonces ha asignado a cada vértice uno de dos colores y ha comprobado que ninguna arista une dos vértices del mismo color. Por tanto, tomando como V_1 los vértices de color 1 y como V_2 los vértices de color 2, obtenemos una partición de V tal

que todas las aristas van de una parte a la otra. Luego G es bipartito.

Si el algoritmo rechaza, entonces ha encontrado una arista $(u, v) \in E$ cuyos extremos tienen el mismo color. Como los colores se han asignado alternando según la distancia desde el vértice inicial de la componente, que u y v tengan el mismo color significa que hay caminos desde el vértice inicial hasta u y hasta v de la misma paridad. Al añadir la arista (u, v) se obtiene un ciclo de longitud impar (esto se ha visto en el razonamiento del recíproco en (b)). Por el apartado (a), G no es bipartito.

Ejercicio 2.1.3. Demostrar que $\mathbf{P}$ es cerrada para la unión y la intersección.

Sean $L_1, L_2 \in \mathbf{P}$. Por definición de $\mathbf{P}$, existen dos máquinas de Turing deterministas M_1 y M_2 que deciden L_1 y L_2 , respectivamente, en tiempo polinómico. Es decir, existen polinomios p_1 y p_2 tales que, para toda palabra x , M_1 decide si $x \in L_1$ en tiempo a lo sumo $p_1(|x|)$, y M_2 decide si $x \in L_2$ en tiempo a lo sumo $p_2(|x|)$. Veamos primero que $L_1 \cup L_2 \in \mathbf{P}$. Construimos una máquina determinista $M_\cup$:

Algoritmo 27 MT $M_\cup$

Entrada: Una palabra x

Ejecuta M_1 sobre x .

Si M_1 acepta **entonces**

$M_\cup$ acepta.

Fin Si

Ejecuta M_2 sobre x .

Si M_2 acepta **entonces**

$M_\cup$ acepta.

Si no

$M_\cup$ rechaza.

Fin Si

Entonces $M_\cup$ acepta x si y solo si M_1 acepta x o M_2 acepta x , es decir, si y solo si $x \in L_1$ o $x \in L_2$. Por tanto, $L(M_\cup) = L_1 \cup L_2$.

Además, su tiempo de ejecución está acotado por

$$p_1(|x|) + p_2(|x|),$$

que es un polinomio. Luego $L_1 \cup L_2 \in \mathbf{P}$. Veamos ahora que $L_1 \cap L_2 \in \mathbf{P}$. Construimos una máquina determinista $M_\cap$ (28). Entonces $M_\cap$ acepta x si y solo si M_1 acepta x y M_2 acepta x , es decir, si y solo si $x \in L_1$ y $x \in L_2$. Por tanto,

$$L(M_\cap) = L_1 \cap L_2.$$

Además, su tiempo de ejecución está acotado por

$$p_1(|x|) + p_2(|x|),$$

que también es un polinomio. Luego $L_1 \cap L_2 \in \mathbf{P}$.

Algoritmo 28 MT $M_{\cap}$

Entrada: Una palabra x
 Ejecuta M_1 sobre x .
Si M_1 rechaza **entonces**
 $M_{\cap}$ rechaza.
Fin Si
 Ejecuta M_2 sobre x .
Si M_2 acepta **entonces**
 $M_{\cap}$ acepta.
Si no
 $M_{\cap}$ rechaza.
Fin Si

Ejercicio 2.1.4. Sea C una clase de funciones de los enteros no negativos en los enteros no negativos. Se dice que C es cerrada para la composición polinómica por la izquierda si $f(n) \in C$ implica que $p(f(n)) = O(g(n))$ para alguna función $g(n) \in C$, donde $p(n)$ es un polinomio cualquiera. Se dice que C es cerrada para la composición polinómica por la derecha si $f(n) \in C$ implica que $f(p(n)) = O(g(n))$ para alguna función $g(n) \in C$, donde $p(n)$ es un polinomio cualquiera.

Determinar cuáles de las siguientes clases de funciones son cerradas para la composición polinómica por la derecha y cuáles lo son por la izquierda.

Nota: en los dos últimos casos se entiende que las funciones se redondean al entero más cercano. En vez de considerar todos los polinomios, como estamos hablando de órdenes de complejidad, será suficiente considerar los polinomios de la forma $p(n) = n^l$, donde l es un número natural.

(a) $C = \{n^k : k > 0\}$.

Sea $f(n) = n^k \in C$ y sea $p(n) = n^l$. Por la izquierda,

$$p(f(n)) = (n^k)^l = n^{kl}.$$

Como $kl > 0$, tomando $g(n) = n^{kl} \in C$, se tiene

$$p(f(n)) = O(g(n)).$$

Luego C es cerrada por la izquierda. Por la derecha,

$$f(p(n)) = f(n^l) = (n^l)^k = n^{lk}.$$

Como $lk > 0$, tomando $g(n) = n^{lk} \in C$, se tiene

$$f(p(n)) = O(g(n)).$$

Luego C es cerrada por la derecha. Por tanto, C es cerrada por la izquierda y por la derecha.

(b) $C = \{kn : k > 0\}$.

Sea $f(n) = kn \in C$. Tomamos $p(n) = n^2$. Entonces, por la izquierda,

$$p(f(n)) = (kn)^2 = k^2n^2.$$

Si existiera $g(n) = an \in C$ tal que $k^2n^2 = O(an)$, entonces tendríamos $n^2 = O(n)$, lo cual es falso. Por tanto, C no es cerrada por la izquierda. Por la derecha,

$$f(p(n)) = f(n^2) = kn^2.$$

Si existiera $g(n) = an \in C$ tal que $kn^2 = O(an)$, entonces tendríamos $n^2 = O(n)$, lo cual es falso. Por tanto, C no es cerrada por la derecha. Por tanto, C no es cerrada ni por la izquierda ni por la derecha.

(c) $C = \{k^n : k > 0\}$.

Sea $f(n) = k^n \in C$ y sea $p(n) = n^l$. Por la izquierda,

$$p(f(n)) = (k^n)^l = k^{ln} = (k^l)^n.$$

Como $k^l > 0$, tomando $g(n) = (k^l)^n \in C$, se tiene

$$p(f(n)) = O(g(n)).$$

Luego C es cerrada por la izquierda. Veamos que no es cerrada por la derecha. Tomamos $f(n) = 2^n \in C$ y $p(n) = n^2$. Entonces

$$f(p(n)) = f(n^2) = 2^{n^2}.$$

Si C fuera cerrada por la derecha, existiría $g(n) = a^n \in C$ tal que

$$2^{n^2} = O(a^n).$$

Pero

$$\frac{2^{n^2}}{a^n} = \left(\frac{2^n}{a}\right)^n \rightarrow \infty,$$

para cualquier constante $a > 0$. Por tanto, 2^{n^2} no es $O(a^n)$ para ningún $a > 0$. Luego C es cerrada por la izquierda, pero no por la derecha.

(d) $C = \{2^{n^k} : k > 0\}$.

Sea $f(n) = 2^{n^k} \in C$ y sea $p(n) = n^l$. Por la izquierda,

$$p(f(n)) = (2^{n^k})^l = 2^{ln^k}.$$

Como

$$\lim_{n \rightarrow \infty} \frac{ln^k}{n^{k+1}} = \lim_{n \rightarrow \infty} \frac{l}{n} = 0,$$

se tiene

$$2^{ln^k} = O(2^{n^{k+1}}).$$

Tomando $g(n) = 2^{n^{k+1}} \in C$, se deduce que C es cerrada por la izquierda. Por la derecha,

$$f(p(n)) = f(n^l) = 2^{(n^l)^k} = 2^{n^{lk}}.$$

Como $lk > 0$, tomando $g(n) = 2^{n^{lk}} \in C$, se tiene

$$f(p(n)) = O(g(n)).$$

Luego C es cerrada por la derecha. Por tanto, C es cerrada por la izquierda y por la derecha.

(e) $C = \{\log^k(n) : k > 0\}$.

Sea $f(n) = \log^k(n) \in C$ y sea $p(n) = n^l$. Por la izquierda,

$$p(f(n)) = (\log^k(n))^l = \log^{kl}(n).$$

Como $kl > 0$, tomando $g(n) = \log^{kl}(n) \in C$, se tiene

$$p(f(n)) = O(g(n)).$$

Luego C es cerrada por la izquierda. Por la derecha,

$$f(p(n)) = f(n^l) = \log^k(n^l) = (l \log(n))^k = l^k \log^k(n).$$

Por tanto,

$$f(p(n)) = O(\log^k(n)).$$

Tomando $g(n) = \log^k(n) \in C$, se deduce que C es cerrada por la derecha. Por tanto, C es cerrada por la izquierda y por la derecha.

(f) $C = \{\log(n)\}$.

Sea $f(n) = \log(n)$. Tomamos $p(n) = n^2$. Por la izquierda,

$$p(f(n)) = (\log(n))^2.$$

Si C fuera cerrada por la izquierda, tendría que cumplirse

$$(\log(n))^2 = O(\log(n)),$$

lo cual es falso. Por tanto, C no es cerrada por la izquierda. Por la derecha, sea $p(n) = n^l$. Entonces

$$f(p(n)) = f(n^l) = \log(n^l) = l \log(n).$$

Así,

$$f(p(n)) = O(\log(n)).$$

Como $\log(n) \in C$, se deduce que C es cerrada por la derecha. Por tanto, C es cerrada por la derecha, pero no por la izquierda.

Ejercicio 2.1.5. Sea $L = \{wcw : w \in \{0, 1\}^*\}$. Demostrar que L se puede reconocer en espacio $\log(n)$.

Sea x una palabra de entrada y sea $n = |x|$. Construimos una MT determinista con una cinta de entrada de solo lectura y una cinta de trabajo. La máquina funciona de la siguiente forma:

Algoritmo 29 MT M

Entrada: Una palabra $x \in \{0, 1, c\}^*$

Calcula $n = |x|$ usando un contador binario.

Recorre x y comprueba que aparece exactamente un símbolo c .

Si no aparece exactamente un símbolo c **entonces**

M rechaza.

Fin Si

Sea k la posición en la que aparece c .

Si $n \neq 2k - 1$ **entonces**

M rechaza.

Fin Si

Para $i = 1, \dots, k - 1$ **hacer**

Lee el símbolo de x en la posición i y lo guarda.

Lee el símbolo de x en la posición $k + i$ y lo compara con el anterior.

Si los símbolos son distintos **entonces**

M rechaza.

Fin Si

Fin Para

M acepta.

Veamos que el algoritmo es correcto. Si M acepta, entonces x tiene exactamente un símbolo c , situado en una posición k tal que $n = 2k - 1$. Por tanto, a la izquierda y a la derecha de c hay exactamente $k - 1$ símbolos. Además, para cada $i = 1, \dots, k - 1$, el símbolo en la posición i coincide con el símbolo en la posición $k + i$. Luego ambas mitades son iguales, y por tanto $x = wcw$ para cierta palabra $w \in \{0, 1\}^*$.

Recíprocamente, si $x \in L$, entonces $x = wcw$ para alguna palabra $w \in \{0, 1\}^*$. En ese caso, x tiene exactamente un símbolo c , las dos partes a izquierda y derecha de c tienen la misma longitud, y todos los símbolos correspondientes coinciden. Por tanto, el algoritmo no rechaza en ningún paso y acepta.

Finalmente, analizamos el espacio. La máquina solo necesita guardar un número constante de contadores, como n , k , i y el contador auxiliar usado para moverse hasta una posición concreta de la entrada. Todos esos contadores están acotados por n , luego cada uno ocupa $O(\log n)$ casillas. Además, solo se guarda un número constante de símbolos de $\{0, 1, c\}$. Por tanto, el espacio total usado en la cinta de trabajo es $O(\log n)$. Concluimos que

$$L \in \mathbf{ESPACIO}(\log n).$$

Ejercicio 2.1.6. Demostrar que si L está en $\mathbf{P}$, entonces L^* también está en $\mathbf{P}$.

Como $L \in \mathbf{P}$, existe un algoritmo determinista A que decide L en tiempo polinómico. Sea $p(n)$ un polinomio tal que A decide si una palabra de longitud n pertenece a L en tiempo $O(p(n))$. Construimos un algoritmo determinista para decidir L^* . Sea $x = x_1 \cdots x_n$ la palabra de entrada. Usaremos programación dinámica. Definimos, para cada $i = 0, \dots, n$,

$$D[i] = 1 \iff x_1 \cdots x_i \in L^*.$$

Inicializamos $D[0] = 1$, ya que $\varepsilon \in L^*$. Para calcular $D[i]$, con $i \geq 1$, usamos la equivalencia

$$x_1 \cdots x_i \in L^* \iff \exists j \in \{0, \dots, i-1\} \text{ tal que } D[j] = 1 \text{ y } x_{j+1} \cdots x_i \in L.$$

Por tanto, el algoritmo es:

Algoritmo 30 Decisor para L^*

Entrada: Una palabra $x = x_1 \cdots x_n$
 $D[0] \leftarrow 1$
Para $i = 1, \dots, n$ **hacer**
 $D[i] \leftarrow 0$
 Para $j = 0, \dots, i-1$ **hacer**
 Si $D[j] = 1$ y A acepta $x_{j+1} \cdots x_i$ **entonces**
 $D[i] \leftarrow 1$
 Fin Si
 Fin Para
Fin Para
Si $D[n] = 1$ **entonces**
 acepta
Si no
 rechaza
Fin Si

Veamos que el algoritmo es correcto. Por definición, $D[0] = 1$, pues $\varepsilon \in L^*$. Supongamos calculados correctamente $D[0], \dots, D[i-1]$. Entonces $D[i] = 1$ si y solo si existe una posición $j < i$ tal que el prefijo $x_1 \cdots x_j$ pertenece a L^* y el bloque final $x_{j+1} \cdots x_i$ pertenece a L . Esto es exactamente decir que $x_1 \cdots x_i$ es concatenación de palabras de L , es decir, que $x_1 \cdots x_i \in L^*$. Por inducción, $D[n] = 1$ si y solo si $x \in L^*$.

Finalmente, analizamos el tiempo. Hay $O(n^2)$ pares (i, j) , y para cada uno se ejecuta el algoritmo A sobre una palabra de longitud menor o igual que n , lo que computacionalmente es de orden $O(p(n))$. Por tanto, el tiempo total está acotado por

$$O(n^2 p(n)),$$

que es polinómico. Luego $L^* \in \mathbf{P}$.

Ejercicio 2.1.7. Demostrar que si L está en $\mathbf{NP}$, entonces L^* también está en $\mathbf{NP}$.

Como $L \in \mathbf{NP}$, existe un verificador determinista V en tiempo polinómico y un polinomio p tales que, para toda palabra u ,

$$u \in L \iff \exists c, |c| \leq p(|u|), \text{ tal que } V(u, c) \text{ acepta.}$$

Veamos que $L^* \in \mathbf{NP}$. Sea $x = x_1 \cdots x_n$ una palabra de entrada. Un certificado de que $x \in L^*$ consiste en una descomposición de x en bloques no vacíos

$$x = u_1 u_2 \cdots u_k,$$

junto con un certificado c_i para cada bloque u_i , demostrando que $u_i \in L$.

Equivalentemente, el certificado contiene posiciones

$$0 = i_0 < i_1 < \cdots < i_k = n,$$

de forma que

$$u_j = x_{i_{j-1}+1} \cdots x_{i_j},$$

y certificados $c_1, \dots, c_k$ tales que $V(u_j, c_j)$ acepta para todo $j = 1, \dots, k$.

El verificador para L^* funciona así:

Algoritmo 31 Verificador para L^*

Entrada: Una palabra $x = x_1 \cdots x_n$ y un certificado formado por $0 = i_0 < i_1 < \cdots < i_k = n$ y palabras $c_1, \dots, c_k$

Si $x = \varepsilon$ **entonces**

 acepta

Fin Si

Para $j = 1, \dots, k$ **hacer**

 Sea $u_j = x_{i_{j-1}+1} \cdots x_{i_j}$

 Ejecuta $V(u_j, c_j)$

Si $V(u_j, c_j)$ rechaza **entonces**

 rechaza

Fin Si

Fin Para

 acepta

Veamos que es correcto. Si $x \in L^*$, entonces existe una descomposición

$$x = u_1 u_2 \cdots u_k$$

con $u_j \in L$ para todo j . Como $L \in \mathbf{NP}$, para cada u_j existe un certificado c_j que hace aceptar a $V(u_j, c_j)$. Por tanto, el verificador anterior acepta x .

Recíprocamente, si el verificador acepta, entonces cada bloque u_j pertenece a L , ya que $V(u_j, c_j)$ acepta. Por tanto,

$$x = u_1 u_2 \cdots u_k \in L^*.$$

Finalmente, veamos que el tamaño del certificado y el tiempo de verificación son polinómicos. Como los bloques son no vacíos, se tiene $k \leq n$. Además, $|u_j| \leq n$,

luego cada certificado c_j tiene tamaño como mucho $p(n)$. Por tanto, el tamaño total del certificado está acotado por

$$O(np(n)),$$

que es polinómico. Además, el verificador ejecuta a lo sumo n veces un algoritmo polinómico sobre palabras de longitud menor o igual que n . Luego el tiempo total también es polinómico.

Así, $L^* \in \mathbf{NP}$.

Ejercicio 2.1.8. Demostrar que $\mathbf{NP}$ es cerrada para la unión y la intersección.

Sean $L_1, L_2 \in \mathbf{NP}$. Por definición de $\mathbf{NP}$, existen dos máquinas de Turing no deterministas M_1 y M_2 que deciden L_1 y L_2 , respectivamente, en tiempo polinómico.

Es decir, existen polinomios p_1 y p_2 tales que, para toda palabra x , M_1 decide si $x \in L_1$ en tiempo a lo sumo $p_1(|x|)$, y M_2 decide si $x \in L_2$ en tiempo a lo sumo $p_2(|x|)$. Veamos primero que $L_1 \cup L_2 \in \mathbf{NP}$. Construimos una máquina de Turing no determinista $M_{\cup}$:

Algoritmo 32 MTND $M_{\cup}$

Entrada: Una palabra x

Elegimos de forma no determinista $i \in \{1, 2\}$.

Si $i = 1$ **entonces**

 Simulamos M_1 sobre x .

Si no

 Simulamos M_2 sobre x .

Fin Si

$M_{\cup}$ acepta si y solo si la máquina simulada acepta.

Entonces $M_{\cup}$ acepta x si y solo si existe una rama de cómputo que acepta en M_1 o existe una rama de cómputo que acepta en M_2 . Por tanto,

$$x \in L(M_{\cup}) \iff x \in L_1 \cup L_2.$$

Además, el tiempo de ejecución de $M_{\cup}$ está acotado por

$$\max\{p_1(|x|), p_2(|x|)\} + O(1),$$

que es un polinomio. Luego $L_1 \cup L_2 \in \mathbf{NP}$.

Veamos ahora que $L_1 \cap L_2 \in \mathbf{NP}$. Construimos una máquina de Turing no determinista $M_{\cap}$:

Esta máquina acepta x si y solo si existe una rama aceptante de M_1 sobre x y, después, existe una rama aceptante de M_2 sobre x . Por tanto,

$$x \in L(M_{\cap}) \iff x \in L_1 \cap L_2.$$

Además, el tiempo de ejecución está acotado por

$$p_1(|x|) + p_2(|x|),$$

que es un polinomio. Luego $L_1 \cap L_2 \in \mathbf{NP}$.

Algoritmo 33 MTND $M_\cap$

Entrada: Una palabra x
 Simulamos M_1 sobre x .
Si M_1 rechaza **entonces**
 $M_\cap$ rechaza.
Fin Si
 Simulamos M_2 sobre x .
Si M_2 acepta **entonces**
 $M_\cap$ acepta.
Si no
 $M_\cap$ rechaza.
Fin Si

Ejercicio 2.1.9. Demostrar que si $\mathbf{NP} \neq \mathbf{CoNP}$, entonces $\mathbf{P} \neq \mathbf{NP}$.

Demostraremos el contrarrecíproco, es decir, si $\mathbf{P} = \mathbf{NP}$, entonces $\mathbf{NP} = \mathbf{CoNP}$. Lo probamos por doble contenido.

$\subseteq$) Sea $L \in \mathbf{NP}$. Como $\mathbf{NP} = \mathbf{P}$, se tiene que $L \in \mathbf{P}$, y como $\mathbf{P}$ es cerrada por complementarios, $\bar{L} \in \mathbf{P}$. De nuevo, como $\mathbf{P} = \mathbf{NP}$, se deduce que $\bar{L} \in \mathbf{NP}$. Por definición de $\mathbf{CoNP}$, esto implica que $L \in \mathbf{CoNP}$. Por tanto,

$$\mathbf{NP} \subseteq \mathbf{CoNP}.$$

$\supseteq$) Recíprocamente, sea $L \in \mathbf{CoNP}$. Entonces, por definición de $\mathbf{CoNP}$, se tiene que $\bar{L} \in \mathbf{NP}$. Como $\mathbf{NP} = \mathbf{P}$, resulta que $\bar{L} \in \mathbf{P}$. Usando otra vez que $\mathbf{P}$ es cerrada por complementario, obtenemos que $L \in \mathbf{P}$. Finalmente, como $\mathbf{P} = \mathbf{NP}$, concluimos que $L \in \mathbf{NP}$. Por tanto,

$$\mathbf{CoNP} \subseteq \mathbf{NP}.$$

Ejercicio 2.1.10. Supongamos que una entrada es una palabra de paréntesis. Demostrar que determinar si están emparejados y anidados correctamente está en $\mathbf{L}$. Lo están $((()))$ y $((()))$, pero no lo están $)() ($ ni $((())) ($.

Sea $L = \{w \in \{(\,)\}^* : w \text{ está correctamente emparejada y anidada}\}$. Vamos a demostrar que $L \in \mathbf{L}$ construyendo un algoritmo determinista que decide L usando espacio logarítmico.

Dada una entrada $w \in \{(\,)\}^*$, el Algoritmo (34) recorre w de izquierda a derecha manteniendo un contador c , que representa el número de paréntesis abiertos que todavía no han sido cerrados.

Veamos que el algoritmo es correcto. Si el algoritmo acepta, entonces nunca ha leído un paréntesis cerrado sin que hubiese antes un paréntesis abierto disponible, pues en tal caso se tendría $c = 0$ al leer un símbolo $)$ y el algoritmo rechazaría.

Algoritmo 34 Algoritmo para paréntesis bien anidados

Entrada: Una palabra $w \in \{(\,,\,)\}^*$
 $c \leftarrow 0$
Para cada símbolo a de w , de izquierda a derecha **hacer**
 Si $a = ($ **entonces**
 $c \leftarrow c + 1$
 Fin Si
 Si $a =)$ **entonces**
 Si $c = 0$ **entonces**
 Rechazar
 Si no
 $c \leftarrow c - 1$
 Fin Si
 Fin Si
Fin Para
Si $c = 0$ **entonces**
 Aceptar
Si no
 Rechazar
Fin Si

Además, al final se tiene $c = 0$, luego todos los paréntesis abiertos han sido cerrados. Por tanto, w está correctamente emparejada y anidada.

Recíprocamente, si w está correctamente emparejada y anidada, entonces en ningún prefijo de w hay más paréntesis cerrados que abiertos. Por tanto, el contador nunca intenta bajar de 0. Además, al final hay tantos paréntesis abiertos como cerrados, luego $c = 0$. Así, el algoritmo acepta.

Finalmente, si $n = |w|$, el contador c nunca toma valores mayores que n . Por tanto, puede almacenarse usando $\lceil \log(n+1) \rceil$ bits. Aparte de este contador, el algoritmo solo usa espacio constante. Luego el espacio total usado es $O(\log n)$. Por tanto, L se decide en espacio logarítmico, es decir, $L \in \mathbf{L}$.

Ejercicio 2.1.11. Consideremos el problema anterior, pero ahora con dos tipos de paréntesis: $(,)$ y $[,]$. Demostrar que determinar si están bien escritos también está en $\mathbf{L}$. Aquí $([] ())$ está permitido, pero $([])$ no. Demostrar que si la entrada se lee de izquierda a derecha sin volver hacia atrás, requieren $O(n)$ de memoria.

Sea $L = \{w \in \{(\,,\,), [\,,\,]\}^* : w \text{ está bien escrita}\}$. Primero comprobamos, como en el ejercicio anterior, que w está correctamente emparejada ignorando el tipo de paréntesis. Es decir, recorremos la entrada con un contador c : sumamos 1 al leer $($ o $[$, restamos 1 al leer $)$ o $]$, rechazamos si c intenta hacerse negativo, y al final exigimos $c = 0$. Esto usa espacio $O(\log n)$.

Falta comprobar que cada apertura se cierra con el símbolo del mismo tipo. Para ello, para cada posición i de la entrada que contenga un símbolo de apertura, bus-

camos su cierre correspondiente. Recorremos la entrada desde la posición $i + 1$ hacia la derecha con un contador c inicializado a 1. Cada vez que leemos una apertura aumentamos c , y cada vez que leemos un cierre disminuimos c . El primer símbolo que hace que $c = 0$ es el cierre correspondiente a la apertura situada en i .

Así, si en la posición i hay $($, comprobamos que dicho cierre sea $)$; si en la posición i hay $[$, comprobamos que dicho cierre sea $]$. Si alguna comprobación falla, rechazamos. Si todas son correctas, aceptamos.

El algoritmo es correcto porque, una vez comprobado que la palabra está bien emparejada ignorando tipos, cada símbolo de apertura tiene un único símbolo de cierre correspondiente. Por tanto, basta comprobar que ambos son del mismo tipo.

En cuanto al espacio, solo se necesita guardar la posición i , el contador c y una cantidad constante de información auxiliar. Como $i \leq n$ y $c \leq n$, ambos se almacenan con $O(\log n)$ bits. Por tanto, el espacio total usado es $O(\log n)$, y se concluye que $L \in \mathbf{L}$.

Por último, si la entrada se lee de izquierda a derecha sin volver hacia atrás, se necesita guardar los tipos de paréntesis abiertos todavía no cerrados. Esto se hace con una pila. En el peor caso, la entrada puede comenzar con $n/2$ símbolos de apertura, por ejemplo una palabra de la forma

$$x_1 x_2 \cdots x_{n/2},$$

donde cada x_i es $($ o $[$. Para poder comprobar después que los cierres aparecen en el orden y con el tipo correcto, hay que recordar esos $n/2$ tipos. Luego se requiere memoria $O(n)$.

Ejercicio 2.1.12. Demostrar que el problema del palíndromo requiere $O(n)$ de memoria si no se puede volver hacia atrás.

En realidad, lo que se demuestra es que requiere memoria lineal, es decir, $\Omega(n)$, y como existe un algoritmo que usa $O(n)$ memoria, el problema requiere $\Theta(n)$ memoria.

Consideremos palabras sobre el alfabeto $\{0, 1\}$ y supongamos que existe un algoritmo que lee la entrada una sola vez, de izquierda a derecha, sin volver hacia atrás, y que decide si una palabra es palíndroma usando $s(n)$ bits de memoria.

Tomemos palabras de longitud $n = 2m$. Después de leer los primeros m símbolos, el algoritmo ha leído una palabra $u \in \{0, 1\}^m$. Hay 2^m posibles palabras u .

Si el algoritmo usara menos de m bits de memoria, entonces tendría menos de 2^m configuraciones posibles de memoria. Por el principio del palomar, existirían dos palabras distintas $u, v \in \{0, 1\}^m$ tales que, después de leer u y después de leer v , el algoritmo queda exactamente en la misma configuración de memoria.

Como el algoritmo no puede volver hacia atrás, a partir de ese momento se comportará igual ante cualquier continuación. En particular, consideremos la continuación u^R , donde u^R es la palabra u escrita al revés.

Entonces:

$$uu^R$$

es un palíndromo, pero

$$vu^R$$

no es un palíndromo, ya que $v \neq u$.

Sin embargo, el algoritmo llega a la misma configuración tras leer u y v , y después recibe la misma continuación u^R . Por tanto, daría la misma respuesta para uu^R y para vu^R , contradicción.

Luego el algoritmo necesita distinguir las 2^m posibles primeras mitades, por lo que necesita al menos m bits de memoria. Como $n = 2m$, esto implica que necesita

$$\Omega(n)$$

memoria.

Por otro lado, existe un algoritmo que guarda toda la palabra, o al menos su primera mitad, y luego compara con la segunda mitad. Este algoritmo usa $O(n)$ memoria.

Por tanto, si la entrada se lee de izquierda a derecha sin volver hacia atrás, el problema del palíndromo requiere memoria lineal:

$$\Theta(n).$$

Ejercicio 2.1.13. Demostrar que $\mathbf{NP} \neq \mathbf{ESPACIO}(n)$. Nota: Este ejercicio es muy complicado. Como sugerencia, para probar que las clases son diferentes, probar que $\mathbf{NP}$ cumple la propiedad de ser cerrada para reducciones en L (si $P_1 \propto P_2$ y P_2 está en $\mathbf{NP}$, entonces P_1 está en $\mathbf{NP}$), mientras que $\mathbf{ESPACIO}(n)$, no.

Sea P_* la propiedad “ser una clase de complejidad cerrada por reducciones en L ”. Recordemos que $\mathbf{L} = \mathbf{ESPACIO}(\log(n))$ (clase de espacio logarítmico determinista). La demostración consiste en probar dos cosas:

1. $\mathbf{NP}$ cumple P_* .
2. $\mathbf{ESPACIO}(n)$ no cumple P_* .

Consecuentemente, deberá ser $\mathbf{NP} \neq \mathbf{ESPACIO}(n)$, ya que si fuera $\mathbf{NP} = \mathbf{ESPACIO}(n)$, en particular, ambas clases deberían cumplir P_* , cosa que vamos a concluir que no sucede.

Consideramos la siguiente definición y resultados antes de demostrar:

Definición 2.1. Un problema de decisión $P_1(x)$ es reducible a un problema de decisión $P_2(y)$ donde $x \in X$ e $y \in Y$, denotado por $P_1(x) \propto P_2(y)$ si y solo si existe un algoritmo determinista que en espacio logarítmico calcula una función $\text{REDU} : X \rightarrow Y$ de tal manera que $P_1(x) = P_2(\text{REDU}(x))$.

Lema 2.1. Sea $\text{REDU} : X \rightarrow Y$ una función calculable por una MT determinista en espacio $O(\log n)$, con $n = |x|$ es la longitud de la entrada $x \in X$, con una cinta de entrada de solo lectura, una cinta de trabajo y una cinta de salida de solo escritura que no vuelve hacia atrás en ningún momento. Entonces $\text{REDU}(x)$ se calcula en tiempo polinómico y existe $d \in \mathbb{N} : |\text{REDU}(x)| \leq n^d$.

Demostración. Sea M una MT determinista que calcula REDU usando espacio $O(\log n)$. Una configuración de M queda determinada por:

$$(q, i, u, v)$$

donde q es el estado, i es la posición del cabezal en la cinta de entrada, y u, v describen el contenido de la cinta de trabajo antes y después del cabezal. Como M usa espacio $O(\log n)$, existe una constante $c > 0$ tal que la longitud total de u y v es menor o igual que $c \log n$. Por tanto, el número de configuraciones posibles está acotado por

$$|Q| \cdot n \cdot |B|^{c \log n}.$$

Como $|Q|$ y $|B|$ son constantes de la máquina,

$$|Q| \cdot n \cdot |B|^{c \log n} \quad \text{tiene orden} \quad O(n \cdot |B|^{c \log n}).$$

Ahora usando que, para $a > 0$ y $n > 0$ se tiene que $a^{\log(n)} = n^{\log(a)}$, entonces, si $a = |B| > 0$:

$$|B|^{c \log n} = (|B|^{\log(n)})^c = (n^{\log |B|})^c = n^{c \log |B|},$$

luego

$$|Q| \cdot n \cdot |B|^{c \log n} \quad \text{tiene orden} \quad O(n \cdot n^{c \log |B|}) = O(n^d).$$

para alguna constante $d \in \mathbb{N}$. Es decir, M tiene solo un número polinómico de configuraciones posibles. Como M es determinista y siempre termina (por ser REDU calculable total), no puede repetir una configuración. Si repitiese una configuración, entraría en un ciclo y no terminaría. Por tanto, el número de pasos de cálculo de M está acotado por el número de configuraciones posibles, que es polinómico. Finalmente, en cada paso M puede escribir como mucho un símbolo en la cinta de salida. Luego, si el número de pasos es polinómico, la longitud de la salida también es polinómica. Por tanto,

$$|\text{REDU}(x)| \leq n^d$$

y REDU se calcula en tiempo polinómico (ya que la calcula la MT M , que solo da un número de pasos polinómico). $\square$

Teorema 2.2 (Teorema de Jerarquía en Espacio). *Si $f(n)$ es una función de complejidad propia, entonces*

$$\mathbf{ESPACIO}(f(n)) \subsetneq \mathbf{ESPACIO}(f(n) \log f(n))$$

Demostración.

1. Para ver que $\mathbf{NP}$ cumple P_* , hay que ver que, dados P_1 y P_2 dos problemas, tal que $P_2 \in \mathbf{NP}$ y $P_1 \propto P_2$, entonces $P_1 \in \mathbf{NP}$. Como $P_1 \propto P_2$, por la Definición 2.1 sabemos que existe una reducción $\text{REDU} : X \rightarrow Y$ calculable por un algoritmo determinista en espacio logarítmico, tal que para toda entrada $x \in X$, entonces $P_1(x) = P_2(\text{REDU}(x))$. Como $P_2 \in \mathbf{NP}$, existe un algoritmo no determinista ALG_2 que decide P_2 en tiempo polinómico. Es decir, existe un polinomio p tal que, para cualquier entrada $y \in Y$, $\text{ALG}_2(y)$ termina en, a lo sumo, tiempo $p(|y|)$. Podemos construir entonces el siguiente algoritmo no

determinista (ya que delega en ALG_2 , que no es determinista) ALG_1 para decidir P_1 :

$$\begin{aligned} \text{ALG}_1(x) : \\ y &:= \text{REDU}(x) \\ \text{return } &\text{ALG}_2(y) \end{aligned}$$

Debemos ver que ALG_1 responde correctamente en todos los casos, y que ALG_1 tarda tiempo polinómico:

- a) Lo primero es trivial, ya que $P_1(x) = P_2(\text{REDU}(x)) = P_2(y)$. Por tanto, ALG_1 acepta x si y solo si ALG_2 acepta y , y eso ocurre si y solo si $P_2(y) = \text{SÍ}$, lo cual a su vez equivale a que $P_1(x) = \text{SÍ}$. Lo mismo pasa con NO y no aceptar.
- b) Para ver que ALG_1 tarda tiempo polinómico, como REDU es calculable en espacio logarítmico, por el Lema 2.1, $\text{REDU}(x)$ se calcula en tiempo polinómico, y además existe $d \in \mathbb{N}$ tal que $|\text{REDU}(x)| \stackrel{(*)}{\leq} n^d$, con $n = |x|$. Por definición de ALG_1 , $y = \text{REDU}(x)$, de donde $\text{ALG}_2(y)$ tarda a lo sumo $p(|y|) = p(|\text{REDU}(x)|)$. Como p es un polinomio,

$$\exists m \in \mathbb{N}, \quad C_t > 0 : t \in \mathbb{N} \implies p(t) \leq C_t t^m$$

luego tenemos que

$$p(|\text{REDU}(x)|) \leq C |\text{REDU}(x)|^m \stackrel{(*)}{\leq} C (n^d)^m = C n^{dm}$$

y $C n^{dm}$ también es un polinomio en la longitud de la entrada $n = |x|$.

Por lo tanto, el tiempo total de ALG_1 es la suma del tiempo en calcular $\text{REDU}(x)$, que por el Lema 2.1 es de orden polinómico, y el tiempo para ejecutar $\text{ALG}_2(\text{REDU}(x))$, que, como acabamos de ver, es también de orden polinómico.

Tenemos entonces que ALG_1 es un algoritmo no determinista que decide correctamente P_1 en tiempo polinómico. Luego $P_1 \in \mathbf{NP}$. De la arbitrariedad de P_1 y P_2 con $P_2 \in \mathbf{NP}$ y $P_1 \propto P_2$, se deduce que $\mathbf{NP}$ cumple P_* .

- 2. Para ver que $\mathbf{ESPACIO}(n)$ no cumple P_* , basta encontrar dos problemas P_1 y P_2 tales que: $P_2 \in \mathbf{ESPACIO}(n)$ y $P_1 \propto P_2$, pero $P_1 \notin \mathbf{ESPACIO}(n)$.

Usaremos el Teorema 2.2, para la función $f(n) = n$, que es propia (es claramente creciente (de hecho, estrictamente creciente), y además se puede calcular la función $g(u) = f(|u|) = |u|$ en unario usando espacio $O(n)$ (de hecho, $O(\log n) \subseteq O(n)$ usando un contador binario) y tiempo $O(n + n) = O(n)$). Entonces se tiene que

$$\mathbf{ESPACIO}(n) \subsetneq \mathbf{ESPACIO}(n \log n)$$

En tal caso, podemos tomar $P_1 \in \mathbf{ESPACIO}(n \log n) \setminus \mathbf{ESPACIO}(n)$. Definimos P_2 como sigue:

$$P_2 = \{x \# 1^{|x|^2} : x \in P_1\}$$

Es decir, una entrada de P_2 es positiva si tiene la forma $x\#1^{|x|^2}$ y además x es una entrada positiva de P_1 . Probamos lo que se quiere en el mismo orden en que se ha enunciado:

- a) $P_2 \in \mathbf{ESPACIO}(n)$. Sea $z = x\#1^{|x|^2}$ una entrada de P_2 , y sea $N = |z|$. Si $m = |x|$, entonces $N = m + 1 + m^2$. Si $m = 0$, entonces $z = \#$, entonces $N = 1$, y el espacio requerido es constante $O(1) \subseteq O(N)$, y en particular estaría en $\mathbf{ESPACIO}(n)$ (aceptamos si $\varepsilon \in P_1$ y rechazamos si $\varepsilon \notin P_1$). Podemos suponer entonces que $m > 0$. Como $P_1 \in \mathbf{ESPACIO}(n \log n)$, decidir si $x \in P_1$ requiere espacio $O(m \log m)$. Pero, para todo $m > 0$,

$$m \log m \leq m^2 \leq m + 1 + m^2 = N$$

Por lo tanto, $O(m \log m) \subseteq O(N)$. Así, la simulación de ALG'_1 (que existe porque $P_1 \in \mathbf{ESPACIO}(n \log n)$) que decide si $x \in P_1$ usa espacio lineal respecto al tamaño N de entrada de P_2 . Además, comprobar que la entrada es de la forma $x\#1^{|x|^2}$ también se puede hacer en espacio $O(N)$ (recorriendo z y usando contadores binarios acotados por N , luego que usen espacio $O(\log N) \subseteq O(N)$). Por tanto, $P_2 \in \mathbf{ESPACIO}(n)$.

- b) $P_1 \propto P_2$. Por definición de reducción, debemos construir una función $\text{REDU}' : X \rightarrow Y$ tal que $x \in P_1 \iff \text{REDU}'(x) \in P_2$, y además debe ser calculable en espacio logarítmico. Para ello, definimos

$$\text{REDU}'(x) = x\#1^{|x|^2}$$

Para que sea calculable en espacio logarítmico, busquemos que se escriba la salida símbolo a símbolo directamente sin guardar toda la salida en la memoria. El algoritmo sería el siguiente:

Entrada : x
 Copiar x en la cinta de salida
 Escribir $\#$ en la cinta de salida
 Escribir tantos 1's como indique $|x|^2$

La primera operación es $O(1)$ en espacio, ya que no hay que almacenar x , y se va copiando símbolo a símbolo. La segunda también es $O(1)$ en espacio por el mismo motivo para el símbolo $\#$, y para la tercera, se obtiene $n = |x|$ por medio de un contador binario, que usa espacio $O(\log n)$. Luego se usan dos contadores binarios i y j para escribir, para cada par (i, j) , con $1 \leq i, j \leq n$, un 1. Cada contador solo necesita almacenar un número entre 1 y n . En binario, necesitaremos $\log n$ bits como máximo por contador. Como son dos contadores, el orden del espacio será $O(\log n) + O(\log n) = O(\log n)$. Así, REDU' es calculable en espacio logarítmico. Por definición, es claro que REDU' es una reducción de P_1 a P_2 . Consecuentemente, $P_1 \propto P_2$.

Como por definición $P_1 \notin \mathbf{ESPACIO}(n)$, entonces $\mathbf{ESPACIO}(n)$ no cumple P_* .

Tenemos entonces que $\mathbf{NP}$ cumple P_* pero $\mathbf{ESPACIO}(n)$ no la cumple, luego, por lo que se ha dicho al principio del ejercicio, se tiene que $\mathbf{NP} \neq \mathbf{ESPACIO}(n)$, como se pedía demostrar. $\square$